

Zero-Knowledge Proof-Carrying Data from Accumulation Schemes

Tianyu Zheng¹, Shang Gao¹, and Xun Liu¹

Department of Computing
The Hong Kong Polytechnic University, Hong Kong, China
{tian-yu.zheng, compxun.liu}@connect.polyu.hk
{shang-jason.gao}@polyu.edu.hk

Abstract. Proof-carrying data (PCD) is a powerful cryptographic primitive for ensuring computational integrity in distributed settings. State-of-the-art PCD constructions based on accumulation schemes achieve practical prover efficiency and support a wide range of applications. However, realizing zero-knowledge for accumulation-based PCD remains challenging, particularly for high-degree relations. Existing solutions often incur substantial overhead due to the need for zero-knowledge in both the underlying non-interactive arguments of knowledge (NARKs) and accumulation schemes. In this work, we present new theoretical and practical improvements for zero-knowledge PCD. First, we propose a novel construction that eliminates the need for zero-knowledge NARKs by separating the compliance predicate and accumulation verification, thereby reducing proof size and improving efficiency. Second, we design an efficient zero-knowledge accumulation scheme for special sound protocols, introducing techniques such as masking vectors and zero-knowledge sum-check protocols to ensure privacy with minimal overhead. The theoretical analysis demonstrates that our construction achieves logarithmic proof size and verification time for d -degree NP relations, and outperforms existing solutions in both asymptotic and concrete complexity.

1 Introduction

Proof-Carrying Data (PCD) [CT10] is a foundational cryptographic primitive that empowers mutually distrustful parties to collaboratively perform distributed computations with strong integrity guarantees. By enabling each participating node to attach succinct proofs certifying the correctness of every intermediate state, PCD ensures that the validity of complex, potentially unbounded computations can be efficiently verified at any point. As a generalization of incrementally verifiable computation (IVC) [Val08], PCD has catalyzed advances in diverse applications, including secure enforcement of language semantics [CTV13], verifiable MapReduce computations [CTV15], consensus protocols [BCG21], scalable blockchains [BMRS20b, BMRS20a], zero-knowledge virtual machine [nex24], and verifiable inference [CCYY25].

Recursive vs. Accumulation. There are two main approaches to constructing PCD. The first, exemplified by recursive SNARKs [BCCT13, BSCTV17, COS20], requires the prover at each step to not only prove the computation of the compliance predicate, but also verify the validity of previous proofs. This demands embedding SNARK verification logic into the circuit, which is both theoretically and practically challenging due to the need for sublinear and circuit-friendly verification [CCDW20]. More recently, a practical alternative based on accumulation or folding schemes [BCMS20, BCL⁺21, KS24, ZGGX23, NDC⁺24, BMNW25, BCFW25] have attracted significant attention. Here, the prover performs the compliance predicate computation and defers verification of previous proofs using accumulation schemes, replacing costly SNARK verification with a much more efficient check on the accumulation proof. Consequently, the underlying non-interactive arguments of knowledge (NARKs) need not be succinct, resulting in improved prover efficiency.

Zero-knowledge PCD. As mentioned in previous works [COS20, BCMS20, BCL⁺21], achieving zero-knowledge is essential for PCD in privacy-sensitive applications such as private smart contract, confidential transactions, and anonymous block-listing [BBB⁺18, Dia21, ZGSX23, KKC⁺25]. In the zero-knowledge PCD setting, each node’s output proof should be cryptographically shielded, ensuring that no sensitive information about the witness is leaked to the distrustful next node. This requirement is notably stronger than that of zero-knowledge IVC, which involves only a single prover and typically demands zero-knowledge only for the proof presented to the verifier. We illustrate this difference by Figure 1.

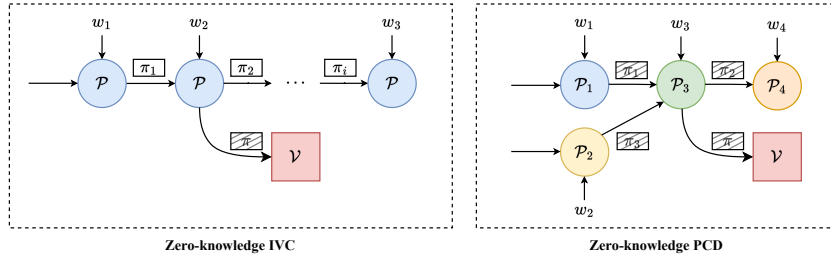


Fig. 1. Comparison of the workflow between zk-IVC and zk-PCD: the zero-knowledge PCD necessitates that every node’s output remain confidential with respect to its witness while the IVC only achieves confidentiality for the proof to the verifier.

Challenges. Despite various accumulation-based PCD solutions, only few of them [BCMS20, BCL⁺21, BMNW25] achieve practical zero-knowledge for low-degree relations such as R1CS. Canonical accumulation-based PCD schemes require both an efficient zero-knowledge NARK and a corresponding zero-knowledge accumulation scheme [BCL⁺21]. In contrast, recursive SNARK-based PCD only

needs the SNARK to be zero-knowledge, for which many efficient solutions exist [BBB⁺18, GWC19, BSBHR19]. Intuitively, achieving zero-knowledge for PCD constructions based on accumulations is inherently more challenging. Practically, implementing zero-knowledge for both the NARK and accumulation schemes faces severe efficiency issues, especially for high-degree relations. This is because standard techniques like random masking introduce exponential overhead—specifically, $O(2^d)$ elements for a NARK proof and $O(m \cdot 2^d)$ for an accumulation proof with m number instances both for d -degree relations in existing solutions [BCL⁺21, BC23, KS24]. Thus, designing efficient zero-knowledge PCD schemes for high-degree relations based on accumulation schemes remains a central challenge in researches.

1.1 Our contributions

Our work makes both theoretical and practical contributions as follows:

- To address the efficiency issues of achieving zero-knowledge for NARKs, we propose a new zero-knowledge PCD construction that directly avoids this requirement in Section 4. Unlike previous approaches, our construction splits the recursive circuit into two parts: the first represents the compliance predicate in terms of the witness, the second represents the accumulation verification without witness. By instantiating the first circuit as a NARK instance-proof pair and then accumulate it by a zero-knowledge accumulation scheme, we observe that has no need to be zero-knowledge. Therefore, we eliminate the significant overhead of achieving zero-knowledge NARKs. We formalize this intuition and provide rigorous proofs: a zero-knowledge PCD can be constructed from a (non-zk) NARK and a zero-knowledge accumulation scheme for the NARK.
- To address the efficiency issues of zero-knowledge accumulation schemes, we present a concrete design for an eligible accumulation scheme for special sound protocols [BC23] in Section 5. Since the zero-knowledge NARK requirement is removed, the accumulation scheme must provide a stronger zero-knowledge property for both the output accumulator and the accumulation proof. Building on previous solutions [BC23, EG23], we make two key modifications:
 - We introduce a masking vector for the final accumulator’s witness. As the witness trivially satisfies the accumulator, this technique adds minimal overhead to the accumulation complexity: the prover and verifier only needs to accumulate one more proof-instance pair.
 - We apply the zero-knowledge sum-check protocol from [CFS17] to provide privacy for the accumulation proof. To address the problem of growing accumulator length caused by the newly generated evaluation claims in the zero-knowledge sum-check, we introduce an additional sum-check protocol running in parallel to update and accumulate these claims.

To the authors’ knowledge, this is the first zero-knowledge accumulation scheme with logarithmic proof size and verification time for d -degree NP relations.

To demonstrate the efficiency of our construction, we provide a comparison in Table 1 of the additional PCD proof size incurred by zero-knowledge between

Table 1. Comparisons of the additional proof size for zk between different PCDs

SPS Protocols	[BCL ⁺ 21]	Ours
Degree- d gate [GWC19]	$O(2^d)\mathbb{G} + O(d \log m)\mathbb{F}$	$2\mathbb{G} + O(d \log m)\mathbb{F}$
Permutation [GWC19]	$4\mathbb{G} + O(\log m)\mathbb{F}$	$2\mathbb{G} + O(\log m)\mathbb{F}$
Lookup [Hab22]	$(4\ell + 2)\mathbb{G} + O(\log m + \ell)\mathbb{F}$	$3\mathbb{G} + O(\log(m + \ell))\mathbb{F}$
R1CS [Gro16]	$7\mathbb{G} + O(\log m)\mathbb{F}$	$2\mathbb{G} + O(\log m)\mathbb{F}$
CCS [STW23]	$O(2^d)\mathbb{G} + O(d \log m)\mathbb{F}$	$2\mathbb{G} + O(d \log m)\mathbb{F}$

Assume m is the number of accumulated predicate instance-proof pairs, d is the maximum degree of relations, ℓ is the size of query set for lookup arguments.

existing zero-knowledge PCD construction [BCL⁺21] and our construction. For the existing construction, we follow the result in [BCL⁺21] and instantiate it with our accumulation scheme in Section 5. This is because no existing work provides zero-knowledge accumulation schemes for high-degree relations. For our scheme, we construct a PCD following the construction given in Section 4 and instantiate it also with our accumulation scheme. We compare their performance i.e., the additional size of the PCD proof output by each node, for different compliance predicates including the custom gate and permutation checks in [GWC19], lookup arguments [Hab22], R1CS relations [Gro16], and CCS relations [STW23]. The designs of special-sound protocol for above predicates are referred to [BC23].

2 Technique Overview

We begin by explaining our first contribution in Section 2.1, a novel construction for zero-knowledge PCD that obviates the need for a zero-knowledge NARK. This construction provides a theoretical advancement that is also applicable to other IVC and PCD frameworks. In Section 2.2, we detail the concrete instantiation of zero-knowledge accumulation schemes, using R1CS relations as a representative example.

2.1 New Construction for Zero-Knowledge PCD

Before presenting our construction, we briefly review how prior work derives PCD from NARKs equipped with accumulation schemes.

Recap of Existing Results. As discussed, zero-knowledge PCD is a versatile primitive with wide-ranging applications. Our starting point is the theorem of [BCL⁺21], which shows how to obtain PCD from any NARK that admits a split accumulation scheme. In broad terms, an accumulation scheme acc for a NARK comprises three algorithms (P, V, D): the prover produces an accumulation proof pf attesting that a new accumulator acc was computed correctly; the verifier checks the local correctness of this computation (i.e., that pf is valid for the

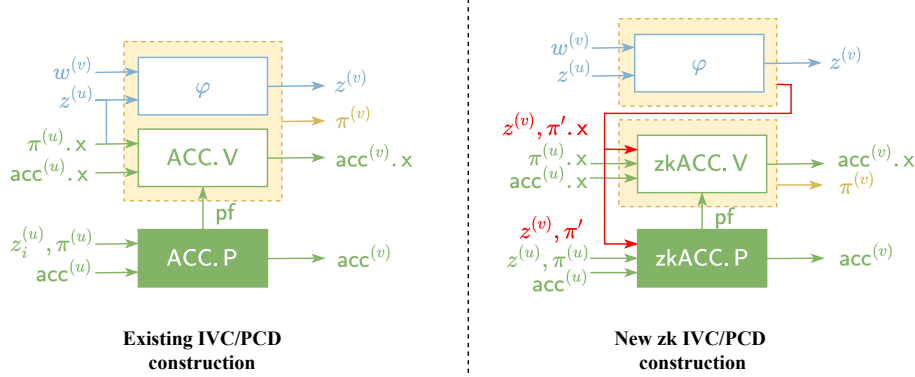


Fig. 2. Comparison between different PCD constructions.

claimed accumulation of acc); and the decider determines the satisfiability of an accumulator acc , i.e., whether acc.w is a satisfying witness to the instance acc.x .

We now describe the PCD construction, illustrated on the left side of Figure 2. The prover at node v performs the following actions:

- **Accumulation.** Given an accumulator $\text{acc}^{(u)}$ and an NARK instance-proof pair $(z^{(u)}, \pi_i)$ from the previous node u , the prover invokes the accumulation proving algorithm ACC.P . This generates a new accumulator $\text{acc}^{(v)}$ and an accumulation proof pf , attesting to the correct transition from $((z^{(u)}, \pi^{(u)}), \text{acc}^{(u)})$ to $\text{acc}^{(v)}$.
- **Recursive Circuit Synthesis.** The prover then constructs a recursive circuit. This circuit takes as extra inputs $z^{(u)}$ and witness $w^{(v)}$, and performs two main checks: (1) it evaluates the PCD predicate $\varphi(z^{(u)}, w^{(v)}, z^{(v)})$ on the claimed inputs, and (2) it verifies the accumulation proof pf by running the logic of the accumulation verifier ACC.V . A crucial feature of a *split* accumulation scheme is that ACC.V only checks on the public instances parts of the accumulated proof as $\pi^{(u)}.x$ and the accumulator as $\text{acc}^{(u)}.x$, without their large witness parts.
- **Proof Generation.** Finally, the prover uses the NARK to generate a new proof $\pi^{(v)}$ for the satisfiability of this recursive circuit. This new proof $\pi^{(v)}$ and the new accumulator $\text{acc}^{(v)}$ are then passed to the next node in the PCD computation.

The final PCD proof consists of the artifacts output in the last step, including the final accumulation proof and the final NARK proof. The PCD verifier checks their satisfiability to ensure the validity of the entire PCD computation. Note that the above process only considers inputs from one previous node u for simplicity, which will be extended to multiple cases in the next part.

A significant drawback of the above construction from [BCL⁺21] is its efficiency: to achieve zero-knowledge for the PCD, it requires both the underlying NARK and the accumulation scheme to be zero-knowledge. The primary bottleneck lies in making the NARK component zero-knowledge. Specifically, the

NARK proof generated from the recursive circuit is *neither succinct nor zero-knowledge*. Hiding this witness with the NARK proof requires applying zero-knowledge mechanisms, e.g., adding a masking string of the same length to the witness. For many high-degree argument systems, such as Plonkish [GWC19] and Customizable Constraint Systems (CCS) [STW23], compiling the relation to be zero-knowledge incurs a substantial overhead. For a relation of degree d , this can introduce $O(2^d)$ new variables and constraints (e.g., via cross-term multiplication), leading to an exponential blowup in the size of the recursive circuit, and significantly limits the practicality of the PCD. Given that there are no known methods to efficiently enforce zero-knowledge for *non-succinct* argument systems with high degree, our work pursues an alternative path: we propose a new construction that eliminates the need for a zero-knowledge NARK.

New Zero-Knowledge Construction. We begin by formalizing the recursive circuit used in prior PCD constructions (some parameters are omitted for clarity). This circuit is designed to prove the validity of a single PCD step, which we denote as step for node v . Now Assume the node v takes inputs from m nodes (also denoted as arity), he then synthesizes a circuit with public inputs including the predecessor outputs $[z^{(u_i)}]_{i=1}^m$, their corresponding NARK proof instances $[\pi^{(u_i)}.\mathbf{x}]_{i=1}^m$, their accumulator instances $[\mathbf{acc}^{(u_i)}.\mathbf{x}]_{i=1}^m$, and the new accumulator instance $\mathbf{acc}^{(v)}.\mathbf{x}$ with proof $\mathbf{pf}$. The circuit also takes private inputs as a witness $w^{(v)}$. As shown in R below, the circuit performs two main checks. First, it checks that the predicate by evaluating $\varphi(z^{(v)}, w^{(v)}, [z^{(u_i)}]_{i=1}^m) = 1$, where φ is the function of PCD’s compliance predicate. Second, the circuit verifies that the new accumulator $\mathbf{acc}^{(v)}$ was correctly derived from the previous ones. Specifically, it runs the logic of the accumulation verifier $\mathbf{ACC.V}$ on the inputs $([\mathbf{acc}^{(u_i)}.\mathbf{x}], \pi^{(u_i)}.\mathbf{x}]_{i=1}^m, \mathbf{acc}^{(v)}.\mathbf{x})$. We present the description as follows:

$R(z^{(v)}, \mathbf{acc}^{(v)}.\mathbf{x}; w^{(v)}, [z^{(u_i)}, \pi^{(u_i)}.\mathbf{x}, \mathbf{acc}^{(u_i)}.\mathbf{x}]_{i=1}^m, \mathbf{pf}):$
1: Check that the compliance predicate $\varphi(z^{(v)}, w^{(v)}, z^{(u_1)}, \dots, z^{(u_m)})$ accepts. 2: Check that the accumulation verifier accepts if $\exists z^{(u_i)} \neq \perp$: $\mathbf{ACC.V}([z^{(u_i)}, \pi^{(u_i)}.\mathbf{x}, \mathbf{acc}^{(u_i)}.\mathbf{x}]_{i=1}^m, \mathbf{acc}^{(v)}.\mathbf{x}, \mathbf{pf}) = 1,$ 3: If the above checks hold, output 1; otherwise, output 0.

Our new construction is based on a key observation: the witness $w^{(v)}$ is exclusively used for the predicate compliance check on φ , while the accumulation verification ($\mathbf{ACC.V}$) operates only on public data. This implies that zero-knowledge is only required for the predicate-check part (step 2 in Figure 2) of the computation. This insight allows us to avoid a monolithic, fully zero-knowledge recursive circuit. Instead, we partition the circuit R into two specialized sub-circuits, R_0 and R_1 , corresponding to steps 1 and 2, respectively, which is shown in Figure 2 (right). In essence, we replace the expensive requirement of a ZK-NARK over the entire recursive logic with a much cheaper ZK-NARK over only the small, predicate-specific sub-circuit R_0 . Assuming the accumulation scheme

is zero-knowledge, the final NARK proof $\pi^{(v)}$ generated for circuit R_1 reveals nothing about the original witness $w^{(v)}$, since $w^{(v)}$ is only used within the confines of the ZK-NARK proof π'_v . Thus, the entire PCD construction achieves zero-knowledge.

The performance benefit of our approach is significant. Compared to prior works [BCMS20, BCL⁺21], the primary overhead is accumulating one additional proof (π'_v) in each step. For typical accumulation schemes, this cost is negligible, especially when compared to the exponential savings from avoiding a ZK-NARK over the entire recursive circuit. Furthermore, our technique is broadly applicable. It can be adapted to other recursive proof systems, such as IVC schemes built on folding, like Nova [KST22] and Halo [BDFG21]. The main prerequisite is an accumulation or folding scheme that can efficiently combine multiple instances. Theoretically, our work decouples the requirement of a ZK-NARK from the construction of ZK-PCD. This provides a new pathway to achieving zero-knowledge in incrementally verifiable computation, as stated below. We formalize this insight as Theorem 1 in Section 4.

2.2 Zero-Knowledge Accumulation Schemes

Our second contribution is a concrete instantiation of a zero-knowledge accumulation scheme for high-degree relations with practical efficiency.

Recap of Protogalaxy. To illustrate our approach, we first review an existing accumulation scheme for R1CS as an example (and extend it to high-degree relations in the main body), initially without considering zero-knowledge. Denote Commit as a homomorphic commitment scheme. Given two committed R1CS instance-witness pairs $[(\mathbf{x}_i, C_i; \mathbf{w}_i)]_{i=0}^1$ (also interpreted as instance-proof pairs in an R1CS NARK system), each pair satisfies the following constraints:

$$\text{R1CS}_{\text{cm}}(\text{ck}) := \{ (\mathbf{x}, C; \mathbf{w}) : C = \text{Commit}(\mathbf{w}) \wedge A\mathbf{z} \circ B\mathbf{z} = C\mathbf{z} \}$$

where $\mathbf{x} \in \mathbb{F}^s$, $\mathbf{w} \in \mathbb{F}^t$, and $\mathbf{z} := (1, \mathbf{x}, \mathbf{w})$, with $A, B, C \in \mathbb{F}^{n \times m}$ and $m = s+t+1$. For notational consistency, we define $\text{V}_{\text{R1CS}}(\mathbf{z}) := A\mathbf{z} \circ B\mathbf{z} - C\mathbf{z} = \mathbf{0}^n$ as an algebraic map of maximum degree 2. We now present an informal description of the accumulation scheme for $[(\mathbf{x}_i, C_i; \mathbf{w}_i)]_{i=0}^1$ according to [BC23, EG23]:

- The prover first computes a vector of polynomials:

$$\mathbf{F}(X) = A\mathbf{z}(X) \circ B\mathbf{z}(X) - C\mathbf{z}(X) \in (\mathbb{F}^{\leq 2}[X])^n, \quad (1)$$

where $\mathbf{z}(X) = eq_0(X) \cdot \mathbf{z}_0 + eq_1(X) \cdot \mathbf{z}_1$ and $eq_b(X)$ is an equality function over $b \in \{0, 1\}$, mapping X to 1 if $X = b$ and 0 otherwise.

- The accumulation verifier samples a random $\alpha \leftarrow \mathbb{F}$;
- The prover and verifier engage in n parallel sum-check protocols (as defined in Section 5.1) for the following n statements:

$$\sum_{b \in \{0, 1\}} eq(\alpha, b) \cdot \mathbf{F}(b) = \mathbf{0}^n;$$

- The sum-check protocols yields the proof tr and evaluation claims $F(\beta) = \mathbf{e}$.

Finally, the prover and verifier agree on a new “relaxed” committed R1CS instance-witness pair $(\mathbf{x}, C, E; \mathbf{w})$, where $E := \text{Commit}(\mathbf{e})$. And the prover sends the sum-check transcript tr as an accumulation proof pf . The accumulated elements are defined as follows:

$$\begin{aligned}\mathbf{x} &:= eq_0(\beta) \cdot \mathbf{x}_0 + eq_1(\beta) \cdot \mathbf{x}_1, \\ C &:= eq_0(\beta) \cdot C_0 + eq_1(\beta) \cdot C_1, \\ \mathbf{w} &:= eq_0(\beta) \cdot \mathbf{w}_0 + eq_1(\beta) \cdot \mathbf{w}_1.\end{aligned}$$

The key insight relies on the properties of the equality polynomials $eq_i(X)$, namely that for all $i \neq j \in \{0, 1\}$: (1) $eq_i(b)^d = eq_i(b)$ and (2) $eq_i(b) \cdot eq_j(b) = 0$ for all $b \in \{0, 1\}$. Therefore, for all $b \in \{0, 1\}$, we have:

$$\mathbf{V}(b) = \mathbf{V}(eq_0(b) \cdot \mathbf{z}_0 + eq_1(b) \cdot \mathbf{z}_1) = eq_0(b) \cdot \mathbf{V}(\mathbf{z}_0) + eq_1(b) \cdot \mathbf{V}(\mathbf{z}_1),$$

which justifies the sum-check statement in Equation 1.

Adding Zero-Knowledge. We now consider how to incorporate zero-knowledge into the accumulation scheme described above. If we only require the randomness of the final output accumulator acc , the prover could simply introduce an additional R1CS pair with a randomly sampled witness among the accumulation prover’s inputs. Specifically, the prover would accumulate three R1CS instance-witness pairs $[(\mathbf{x}_i, C_i; \mathbf{w}_i)]_{i=0}^2$, where $\mathbf{x}_2$ is a trivially satisfiable instance vector corresponding to a random witness $\mathbf{w}_2 \leftarrow \mathbb{F}^t$. However, this approach is insufficient, as the accumulation proof pf may still leak information about the witness.

A practical solution is to employ the zero-knowledge sum-check protocol proposed in [CFS17], which achieves zero-knowledge by masking the coefficients of the target n -variate polynomial $F(\mathbf{X}) \in \mathbb{F}^{(\leq d)}[\mathbf{X}]$ with a random polynomial $G(\mathbf{X})$ of the same variables and individual degrees as $F(\mathbf{X})$. Furthermore, as discussed in Section 5.1, Xie et al. optimize this approach by reducing the size of $G(\mathbf{X})$ from $O(n \cdot d)$ to $O(d)$ [XZZ⁺19]. Nevertheless, introducing the zero-knowledge sum-check protocol into accumulation schemes presents a new challenge: the sum-check prover outputs additional evaluation claims of the form $\mathbf{G}(\beta) = \mathbf{v}$ when proving the following statement:

$$\sum_{b \in \{0, 1\}} eq(\alpha, b) \cdot \mathbf{F}(b) + \gamma \cdot \mathbf{G}(b) = \gamma \cdot \mathbf{r}$$

where γ is a challenge and $\mathbf{r} := \sum_{b \in \{0, 1\}} \mathbf{G}(b)$. The difficulty arises because, unlike the claims for $\mathbf{F}(\beta)$, the prover cannot simply defer checking the claim for $\mathbf{G}(\beta)$; otherwise, the length of the accumulator would continue to grow with the number of accumulation steps (i.e., with the number recursive steps in PCD).

To address this issue, we adopt a technique from [KS24], which updates the evaluation point at each new accumulation step. In essence, whenever the prover

generates a new evaluation claim $\mathbf{G}'(\beta') = \mathbf{v}'$, they can also update the point β in the previous claim $\mathbf{G}(\beta) = \mathbf{v}$ to β' by executing another round of the sum-check protocol (this time without zero-knowledge). As a result, the two claims $\mathbf{G}'(\beta')$ and $\mathbf{G}(\beta')$ can be linearly combined without incurring additional overhead.

In summary, we obtain an accumulation scheme between a committed R1CS pair $(\mathbf{x}_0, C_0; \mathbf{w}_0)$ and an accumulator

$$\text{acc} := (\text{acc.x} = (\mathbf{x}_1, C_1, E, \mathbf{v}), \text{acc.w} = (\mathbf{w}_1, \mathbf{G}(X)))$$

where the size of the accumulator is independent of the number of instances being accumulated. In Section 5, we present a more general accumulation scheme for NP relations, which possesses the following features:

- Utilizes special-sound protocols as defined in prior works [BC23, EG23].
- Supports accumulation for multiple predicate pairs and *multiple* accumulators, a capability not achieved in [EG23].
- Achieves zero-knowledge with practical prover efficiency, scaling linearly with the number of input predicate instances.

3 Preliminary

3.1 Notations

In this paper, we use λ to denote the security parameter. Accordingly, $\text{negl}(\lambda)$ denotes an unspecified function that is negligible in λ . We denote by $[n]$ the set $\{1, \dots, n\} \subseteq \mathbb{N}$. Let $\mathbb{F}$ denote a finite field, e.g., $\mathbb{F}_p$ is a prime field for a large prime p . The vector is denoted as $\mathbf{a} \in \mathbb{F}^n$ with elements $a_1, \dots, a_n \in \mathbb{F}$. $\mathbf{a}[i]$ is also used to denote the i -th element of $\mathbf{a}$ when the element is not specified with a concrete value. To represent a set, we use $\{a_i\}_{i=1}^n$ as a short-hand for $\{a_1, \dots, a_n\}$. For a finite set S , let $x \leftarrow S$ denote sampling x from S uniformly at random. We use “PPT algorithms” to refer to “Probabilistic Polynomial Time” algorithms. For a non-deterministic polynomial time (NP) *indexed relation* R parameterized over public parameters $\mathbf{pp}$ (e.g., including the field $\mathbb{F}$), it consists of a triple of index i , instance $\mathbf{x}$, and witness $\mathbf{w}$ (the secret inputs).

3.2 NARK

Specifically, we define a non-interactive argument of knowledge NARK for an indexed relation R as a tuple of algorithms (G, I, P, V) below, each with an access to the random oracle RO:

- $G^{\text{RO}}(1^\lambda) \rightarrow \mathbf{pp}$ the generator takes as inputs the security parameter λ in unary, and outputs the public parameter $\mathbf{pp}$.
- $I^{\text{RO}}(\mathbf{pp}, i) \rightarrow (\text{ipk}, \text{ivk})$ the indexer takes as inputs $\mathbf{pp}$ and an index i under relation R , and outputs a long index proving key ipk and a short index verification key ivk .

- $P^{\text{RO}}(\text{ipk}, x, w) \rightarrow \pi$ the prover takes as inputs the ipk , and a tuple (x, w) under relation R , and outputs a NARK proof π .
- $V^{\text{RO}}(\text{ivk}, x, \pi) \rightarrow b$ the verifier takes as inputs the ivk , and a tuple (x, π) , and outputs a bit b indicating whether π is valid.

NARK is secure in the random oracle model if the completeness and knowledge soundness hold as defined in Appendix C.3.

3.3 Accumulation Schemes

We adopt the definition of the accumulation scheme for NARK given in [BCL⁺21]. Let $\Phi : \{0, 1\}^* \rightarrow \{0, 1\}$ be a (relation) predicate and $\mathcal{H}$ be a randomized algorithm that can be accessed as an oracle, which inputs Φ and outputs predicate parameters pp_Φ . A (split) *accumulation scheme* for $(\Phi, \mathcal{H})$ is a tuple of algorithms $\text{AS} = (G, I, P, V, D)$ of which P, V have access to the same random oracle RO . Typically, we denote an accumulator acc with the same form as a predicate input, consisting of the instance part acc.x and the witness part acc.w , and a decider algorithm D reads the accumulator in its entirety and checks its validity. Generally, these algorithms are expected to satisfy the following security properties:

Definition 1 (Completeness). *ACC is complete if for every (unbounded) adversary $\mathcal{A}$,*

$$\Pr \left[\begin{array}{l} \forall j \in [m], \text{ACC.D}(\text{adk}, \text{acc}_j) = 1 \\ \forall i \in [n], \Phi(\text{pp}_\Phi, i_\Phi, \text{qx}_i, \text{qw}_i) = 1 \\ \Downarrow \\ \text{acc.V}^{\text{RO}} \left(\begin{array}{l} \text{avk}, [\text{qx}_i]_{i=1}^n \\ [\text{acc}_j.\text{x}]_{j=1}^m \\ \text{acc.x}, \text{pf} \end{array} \right) = 1 \\ \text{acc.D}(\text{adk}, \text{acc}) = 1 \end{array} \right] = 1$$

$$\left[\begin{array}{l} \text{RO} \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{ACC.G}(1^\lambda) \\ \text{pp}_\Phi \leftarrow \mathcal{H}(1^\lambda) \\ \left(i_\Phi, [\text{acc}_j]_{j=1}^m, \right) \leftarrow \mathcal{A}^{\text{RO}}(\text{pp}, \text{pp}_\Phi) \\ (\text{apk}, \text{avk}, \text{adk}) \leftarrow \text{ACC.I}(\text{pp}, \text{pp}_\Phi, i_\Phi) \\ (\text{acc}, \text{pf}) \leftarrow \text{ACC.P}^{\text{RO}} \left(\text{apk}, [\text{acc}_j]_{j=1}^m, \right. \\ \left. [(\text{qx}_i, \text{qw}_i)]_{i=1}^n \right) \end{array} \right]$$

Note that for $m = n = 0$, the precondition on the left-hand side holds vacuously, and this is required for the completeness condition to be non-trivial.

Definition 2 (Knowledge Soundness). *ACC is knowledge sound with respect to auxiliary input distribution Ψ if for every (non-uniform) expected polynomial-*

time adversary $\tilde{P}$, there exists an expected polynomial-time extractor $\mathcal{E}$ such that

$$\Pr \left[\begin{array}{l} \text{ACC.V}^{\text{RO}} \left(\begin{array}{l} \text{avk}, [\text{qx}_i]_{i=1}^n, \\ [\text{acc}_j.\text{x}]_{j=1}^m, \\ \text{acc.x}, \text{pf} \end{array} \right) = 1 \\ \wedge \text{ACC.D}(\text{adk}, \text{acc}) = 1 \\ \Downarrow \\ \forall i \in [n], \\ \Phi(\text{pp}_\Phi, i_\Phi, \text{qx}_i, \text{qw}_i) = 1 \\ \forall j \in [m], \\ \text{ACC.D}(\text{adk}, (\text{acc}_j.\text{x}, \text{acc}_j.\text{w})) = 1 \end{array} \middle| \begin{array}{l} \text{RO} \leftarrow \mathcal{U}(\lambda) \\ \text{pp} \leftarrow \text{ACC.G}(1^\lambda) \\ \text{pp}_\Phi \leftarrow \mathcal{H}(1^\lambda) \\ \text{ai} \leftarrow \Psi(1^\lambda) \\ \left(\begin{array}{l} i_\Phi, [\text{qx}_i]_{i=1}^m, \\ [\text{acc}_j.\text{x}]_{j=1}^m, \\ \text{acc}, \text{pf}; r \end{array} \right) \leftarrow \tilde{P}^{\text{RO}}(\text{pp}, \text{pp}_\Phi, \text{ai}) \\ \left(\begin{array}{l} [\text{qw}_i]_{i=1}^n, \\ [\text{acc}_j.\text{w}]_{j=1}^m \end{array} \right) \leftarrow \mathcal{E}^{\tilde{P}, \text{RO}}(\text{pp}, \text{pp}_\Phi, \text{ai}, r) \\ (\text{apk}, \text{avk}, \text{adk}) \leftarrow \text{ACC.I}(\text{pp}, \text{pp}_\Phi, i_\Phi) \end{array} \right] \approx 1$$

3.4 Proof-Carrying Data

According to [BCL⁺21], a Proof-Carrying Data consists of a tuple of algorithms $\text{PCD} = (\text{G}, \text{I}, \text{P}, \text{V})$. We start by defining some necessary terminology and then mention the security definitions.

Definition 3. A transcript T is a directed acyclic graph where each vertex $u \in V(\mathsf{T})$ is labeled by local data $z_{\text{loc}}^{(u)}$ and each edge $e \in E(\mathsf{T})$ is labeled by a message $z^{(e)} \neq \perp$. The output of a transcript T denoted as $\circ(\mathsf{T})$, is a message $z^{(e)}$ where $e = (u, v)$ is the lexicographically-first edge such that v is a sink.

Definition 4. A vertex $u \in V(\mathsf{T})$ is φ -compliant for $\varphi \in \mathsf{F}$ if for all outgoing edges $e = (u, v) \in E(\mathsf{T})$:

- (base case) if u has no incoming edges, $\varphi(z^{(e)}, z_{\text{loc}}^{(u)}, \perp, \dots, \perp)$ accepts;
- (recursive case) if u has incoming edges $e_1, \dots, e_m$, $\varphi(z^{(e)}, z_{\text{loc}}^{(u)}, z^{(e_1)}, \dots, z^{(e_m)})$ accepts.

T is φ -compliant if each of its vertices is φ -compliant.

Next, we define the security properties of PCD. Note that the completeness and zero-knowledge are defined for each possible vertex in a transcript. Hence, for simplicity, we omit superscripts indicating the vertex and edge in $(z^{(e)}, z_{\text{loc}}^{(u)}, z^{(e_1)}, \dots, z^{(e_m)})$.

Definition 5 (Perfect Completeness). PCD has perfect completeness if for every adversary $\mathcal{A}$ the following holds:

$$\Pr \left[\begin{array}{l} \left(\begin{array}{l} \varphi \in \mathsf{F} \\ \wedge \varphi(z, z_{\text{loc}}, z_1, \dots, z_m) = 1 \\ \wedge \forall i, z_i = \perp \vee \text{PCD.V}(\text{ivk}, z_i, \pi_i) = 1 \end{array} \right) \\ \Downarrow \\ \text{PCD.V}(\text{ivk}, z, \pi) = 1 \end{array} \middle| \begin{array}{l} \text{pp} \leftarrow \text{PCD.G}(1^\lambda); \\ (\varphi, z, z_{\text{loc}}, [z_i]_{i=1}^m) \leftarrow \mathcal{A}(\text{pp}); \\ (\text{ipk}, \text{ivk}) \leftarrow \text{PCD.I}(\text{pp}, \varphi); \\ \Pi \leftarrow \text{PCD.P}(\text{ipk}, z, z_{\text{loc}}, [z_i, \pi_i]_{i=1}^m) \end{array} \right] = 1$$

Definition 6 (Knowledge Soundness). PCD has knowledge soundness (w.r.t. an auxiliary input distribution $\mathcal{D}$) if for every expected polynomial time adversary P^* , there exists an expected polynomial time extractor Ext^{P^*} such that for every set Z :

$$\Pr \left[\begin{array}{c} \varphi \in F \\ \wedge (\text{pp}, \text{ai}, \varphi, \circ(\text{T}), \text{ao}) \in Z \\ \wedge \text{T is } \varphi\text{-compliant} \end{array} \middle| \begin{array}{c} \text{pp} \leftarrow \text{PCD.G}(1^\lambda); \\ \text{ai} \leftarrow \mathcal{D}(\text{pp}); \\ (\varphi, \text{T}, \text{ao}) \leftarrow \text{Ext}^{P^*}(\text{pp}, \text{ai}) \end{array} \right],$$

$$\geq \Pr \left[\begin{array}{c} \varphi \in F \\ \wedge (\text{pp}, \text{ai}, \varphi, \circ, \text{ao}) \in Z \\ \wedge \text{PCD.V}(\text{ivk}, \circ, \pi) = 1 \end{array} \middle| \begin{array}{c} \text{pp} \leftarrow \text{PCD.G}(1^\lambda); \\ \text{ai} \leftarrow \mathcal{D}(\text{pp}); \\ (\varphi, \circ, \pi, \text{ao}) \leftarrow P^*(\text{pp}, \text{ai}); \\ (\text{ipk}, \text{ivk}) \leftarrow \text{PCD.I}(\text{pp}, \varphi, F) \end{array} \right].$$

where ai and ao are auxiliary inputs and outputs. These modifications aim to ensure closeness in distribution between the outputs of the prover and the extractor for the strong extraction guarantee [BCCT13]. As a result, we can prove post-quantum security for our PCD as mentioned in Theorem 1.

Definition 7 (Zero-Knowledge). PCD has (statistical) zero-knowledge if there exists a probabilistic polynomial-time simulator Sim such that for every polynomial-size honest adversary $\mathcal{A}$, the distributions below are computationally indistinguishable:

$$\left\{ (\text{pp}, \varphi, z, \pi) \middle| \begin{array}{c} \text{pp} \leftarrow \text{PCD.G}(1^\lambda); \\ (\varphi, z, z_{\text{loc}}, [z_i, \pi_i]_{i=1}^m) \leftarrow \mathcal{A}(\text{pp}); \\ (\text{ipk}, \text{ivk}) \leftarrow \text{PCD.I}(\text{pp}, \varphi, F); \\ \pi \leftarrow \text{PCD.P}(\text{ipk}, z, z_{\text{loc}}, [z_i, \pi_i]_{i=1}^m) \end{array} \right\}$$

and

$$\left\{ (\text{pp}, \varphi, z, \pi) \middle| \begin{array}{c} (\text{pp}, \tau) \leftarrow \text{Sim}(1^\lambda); \\ (\varphi, z, z_{\text{loc}}, [z_i, \pi_i]_{i=1}^m) \leftarrow \mathcal{A}(\text{pp}); \\ \pi \leftarrow \text{Sim}(\tau, \varphi, z) \end{array} \right\}.$$

An adversary is honest if its output satisfies the implicant of the completeness condition with probability 1, namely: $\varphi \in F, \varphi(z, z_{\text{loc}}, z_1, \dots, z_m) = 1$, and either $\forall i, z_i = \perp$ or $\forall i, \text{PCD.V}(\text{ivk}, z_i, \pi_i) = 1$.

4 New Construction for Zero Knowledge PCD

We first formalize our main theorem for constructing zero-knowledge PCD in Section 4.1. Next in Section 4.2, we propose the full construction of zero-knowledge PCD from zero-knowledge accumulation schemes, which proves the main theorem. Finally, we present the security analysis of the proposed PCD construction in Section 4.3.

4.1 Main Theorem

To formalize the theorem, we first present the necessary definition and describe the syntax of accumulation schemes with zero-knowledge property.

Definition 8 (Zero-Knowledge Accumulation Schemes). Denote $\text{NARK} = (\text{G}, \text{I}, \text{P}, \text{V})$ as a non-interactive argument of a knowledge, define $\Phi : \{0, 1\}^* \rightarrow \{0, 1\}$ as a predicate for NARK.V and $\mathcal{H} := \text{NARK.G}$ as a randomized oracle algorithm that outputs predicate parameters pp_Φ , where φ_{NARK} is defined below:

$\varphi_{\text{NARK}}(\text{pp}_\Phi, i_\Phi, \text{qx} = (x, \pi.x), \text{qw} = \pi.w):$
$1: (\text{ipk}, \text{ivk}) \leftarrow \text{NARK.I}(\text{pp}_\Phi, i_\Phi).$
$2: \text{Output } \text{NARK.V}(\text{ivk}, x, (\pi.x, \pi.w)).$

An accumulation scheme for $(\Phi, \mathcal{H})$ in the random oracle model consists of a tuple of algorithms as $\text{ACC} = (\text{G}, \text{I}, \text{P}, \text{V}, \text{D})$, where G is the same as NARK , P, V have access to the same random oracle RO . We describe their syntaxes as follows:

- $\text{ACC.G}(1^\lambda) \rightarrow \text{pp}$: on inputs a security parameter λ in unary, the generator $\text{ACC.G}(1^\lambda)$ samples and outputs public parameters pp .
- $\text{ACC.I}^{\text{RO}}(\text{pp}, \text{pp}_\Phi, i_\Phi) \rightarrow (\text{apk}, \text{avk}, \text{adk})$: on inputs public parameters pp , predicate parameters pp_Φ generated by $\mathcal{H}$ and a predicate index i_Φ , the indexer ACC.I deterministically computes and outputs a triple $(\text{apk}, \text{avk}, \text{adk})$ consisting an accumulator proving key apk , an accumulator verification key (apk, avk) , and a decision key adk .
- $\text{ACC.P}^{\text{RO}}(\text{apk}, [(\text{qx}_i, \text{qw}_i)]_{i=1}^n, [\text{acc}_j]_{j=1}^m) \rightarrow (\text{acc}, \text{pf})$: on inputs the key apk , predicate inputs $[(\text{qx}_i, \text{qw}_i)]_{i=1}^n$, and old accumulators $[\text{acc}_j]_{j=1}^m = [(\text{acc}_j.x, \text{acc}_j.w)]_{j=1}^m$, the prover ACC.P outputs a new accumulator $\text{acc} = (\text{acc.x}, \text{acc.w})$ and a proof pf for the accumulation verifier.
- $\text{ACC.V}^{\text{RO}}(\text{avk}, [\text{qx}_i]_{i=1}^n, [\text{acc}_j.x]_{j=1}^m, \text{acc.x}, \text{pf}) \rightarrow b$: on inputs the key avk , predicate input instances $[\text{qx}_i]_{i=1}^n$, old accumulator instance $[\text{acc}_j.x]_{j=1}^m$, a new accumulator instance acc.x , and a proof pf , the verifier ACC.V outputs a bit indicating whether acc.x correctly accumulated $[\text{qx}_i]_{i=1}^n$ and $[\text{acc}_j.x]_{j=1}^m$.
- $\text{ACC.D}(\text{adk}, \text{acc}) \rightarrow b$: on inputs the key adk , and an accumulator $\text{acc} = (\text{acc.x}, \text{acc.w})$, the decider ACC.D outputs a bit indicating whether acc is a valid accumulator.

The accumulation scheme ACC defined above should satisfy completeness, knowledge soundness given in Definitions 1 and 2, and additionally zero-knowledge as defined below. Note that our definition is a variant of the one given in [BCL⁺21] that only requires the simulator to simulate an indistinguishable accumulator acc without an accumulation proof pf . Since our PCD construction removes the requirement of the zero-knowledge property for NARK , our zero-knowledge accumulation scheme must ensure the privacy of both the accumulator and the accumulation proof.

Definition 9 (Zero Knowledge). Assume a polynomial-time “honest” adversary $\mathcal{A}$ that outputs a tuple $(i_\Phi, [(qx_i, qw_i)]_{i=1}^n, [acc_j]_{j=1}^m)$ with probability 1, such that $\Phi(pp_\Phi, i_\Phi, qx_i, qw_i) = 1$ and $\text{ACC.D}(\text{adk}, acc_j) = 1$ for all $i \in [n]$ and $j \in [m]$. ACC is zero knowledge if there exists a polynomial-time simulator Sim such that the following two distributions are (statistically/computationally) indistinguishable.

$$\left\{ (RO, pp, pp_\Phi, i_\Phi, acc, pf) \left| \begin{array}{l} RO \leftarrow \mathcal{U}(\lambda) \\ pp \leftarrow \text{ACC.G}(1^\lambda) \\ pp_\Phi \leftarrow \mathcal{H}(1^\lambda) \\ (i_\Phi, [(qx_i, qw_i)]_{i=1}^n, [acc_j]_{j=1}^m) \leftarrow \mathcal{A}^{\text{RO}}(pp, pp_\Phi) \\ (\text{apk}, \text{avk}, \text{adk}) \leftarrow \text{ACC.I}(pp, pp_\Phi, i_\Phi) \\ (acc, pf) \leftarrow \text{ACC.P}^{\text{RO}}(\text{apk}, [(qx_i, qw_i)]_{i=1}^n, [acc_j]_{j=1}^m) \end{array} \right. \right\}$$

$$\left\{ (RO[\mu], pp, pp_\Phi, i_\Phi, acc, pf) \left| \begin{array}{l} RO \leftarrow \mathcal{U}(\lambda) \\ (pp, \tau) \leftarrow \text{Sim}^{\text{RO}}(1^\lambda) \\ pp_\Phi \leftarrow \mathcal{H}(1^\lambda) \\ (i_\Phi, [(qx_i, qw_i)]_{i=1}^n, [acc_j]_{j=1}^m) \leftarrow \mathcal{A}^{\text{RO}}(pp, pp_\Phi) \\ (acc, pf, \mu) \leftarrow \text{Sim}^{\text{RO}}(\tau, pp_\Phi, i_\Phi, [qx_i]_{i=1}^n, [acc_j]_{j=1}^m) \end{array} \right. \right\},$$

where $RO[\mu]$ denotes the RO domain-separated/modified to be consistent with the query-answer list μ . For the description of PCD constructions, we propose the following definition for the representation of accumulation verification in recursive circuits.

Definition 10 (Accumulation Verifier Circuit). We say that an accumulation scheme for a non-interactive argument of knowledge NARK is an accumulation scheme $\text{ACC} = (\text{G}, \text{I}, \text{P}, \text{V}, \text{D})$ for the pair $(\Phi, \mathcal{H})$. We denote $V^{\lambda, m, N, k}$ with respect to security parameter λ , the number m of all accumulated predicate inputs and accumulators, the maximum index size N , and the maximum instance size k , as the circuit corresponding to the computation of the accumulation verifier ACC.V . Moreover, we denote $v(\lambda, m, N, k)$ as the size of the circuit $V^{\lambda, m, N, k}$, $|\text{avk}(\lambda, m, N)|$ as the size of the accumulator verification key avk , and $|\text{acc.x}(\lambda, m, N)|$ as the size of an accumulator instance.

Note that the accumulator instance size of acc.x is required to be independent of the instance size bound k . This can be achieved by mapping the original k -length instance acc.x into an image with a collision-resistant hash function in the design of accumulation scheme¹. Correspondingly, the new accumulator instance only contains $H(\text{acc.x})$ and acc.x is taken as the witness.

Next, we formalize this theoretical result in Theorem 1. Different from previous works [BCMS20, BCL⁺21, ZZD23], we introduce a novel construction for zero-knowledge PCD that only depends on the zero-knowledge property of accumulation schemes, thus eliminating the additional cost incurred in achieving zero-knowledge NARKs.

¹ In Nova, such operation is explicitly performed by the IVC/PCD prover and verifier.

Theorem 1 (Zero-Knowledge PCD Construction (adopted from [BCL⁺21])).

There exists a polynomial-time transformation T such that if $\text{NARK} = (G, I, P, V)$ is a (non-zero-knowledge) non-interactive argument of knowledge for circuit satisfiability and ACC is a zero-knowledge accumulation scheme for NARK , then $\text{PCD} = (G, I, P, V) := T(\text{NARK}, \text{ACC})$ is a zero-knowledge Proof-Carrying Data scheme for constant-depth compliance predicates, provided

$$\exists \epsilon \in (0, 1) \text{ and a polynomial } \alpha \text{ s.t. } v^*(\lambda, m, N, \sigma) = O(N^{1-\epsilon} \cdot \alpha(\lambda, m, \sigma)).$$

where v^* is defined as the size of the accumulation verifier invoked in the recursive circuit, where the instance consists of an accumulator verification key avk , an accumulator instance, and some additional data of size σ . Specifically, we have the following equation that is sublinear in N :

$$v^*(\lambda, m, N, \sigma) := v(\lambda, m, N, |\text{avk}(\lambda, m, N)| + |\text{acc.x}(\lambda, m, N)| + \sigma).$$

Moreover, if NARK and ACC are secure against quantum adversaries, then PCD is secure against quantum adversaries. The theoretical performance of PCD is highlighted below, where the size of the predicate $\varphi : \mathbb{F}^{(m+2)\sigma} \rightarrow \mathbb{F}$ is $f = \omega(\alpha(\lambda, m, \sigma)^{1/\epsilon})$:

- the cost of running PCD.I is equal to the cost of running both NARK.I and ACC.I on an index of size f and an index of size $\circ(f)$;
- the cost of running PCD.P is equal to the cost of accumulating $m+1$ instance-proof pairs using ACC.P , and running NARK.P twice, one on an index of size f , and one on an index of size $\circ(f)$.
- the cost of running PCD.V is equal to the cost of running NARK.V on an index of size $\circ(f)$, and running ACC.D on two indexes of size f and $\circ(f)$ respectively.

Note that the theoretical performance of the new PCD construction is slightly larger than the previous construction, since there are two accumulators for different predicates, as explained later. Next, we present the detailed construction of our zero-knowledge PCD as a proof of Theorem 1.

4.2 Construction

Let $\text{NARK} = (G, I, P, V)$ be a (non-zero-knowledge) non-interactive argument of knowledge for circuit satisfiability and ACC is a zero-knowledge accumulation scheme for NARK . For ease of presentation, we slightly abuse the denotation of acc to represent a pair of accumulators, i.e., $\text{acc} := (\text{acc}^{(0)}, \text{acc}^{(1)})$, for different predicates $\Phi^{(0)}, \Phi^{(1)}$ respectively. we refer to acc.x as $(\text{acc}^{(0)}.x, \text{acc}^{(1)}.x)$. We construct a zero-knowledge PCD as $\text{PCD} = (G, I, P, V)$ as follows.

Given a compliance predicate $\varphi : \mathbb{F}^{(m+2)\sigma} \rightarrow \mathbb{F}$, the circuit that realizes the recursion is defined as follows.

$$R_{\varphi}^{(\lambda, N, k)}(z; z_{\text{loc}}, [z_i]_{i=1}^m):$$

- 1: Check that the compliance predicate $\varphi(z, z_{\text{loc}}, z_1, \dots, z_m)$ accepts.
- 2: If the above check holds, output 1; otherwise, output 0.

$R_V^{(\lambda, N, k)}(\text{avk}, \text{acc.x}; [z_i, \pi_i.\text{x}, \text{acc}_i.\text{x}]_{i=1}^m, \text{pf})$:

- 1: If there exists $i \in [m]$ such that $z_i \neq \perp$, check that the verifier accepts

$$V^{(\lambda, m, N, k)}(\text{avk}, [\text{qx}_i]_{i=1}^m, [\text{acc}_i.\text{x}]_{i=1}^m, \text{acc.x}, \text{pf}) = 1,$$
where $\text{qx}_i := ((\text{avk}, z_i, \text{acc}_i.\text{x}), \pi_i.\text{x})$.
- 2: If the above check holds, output 1; otherwise, output 0.

Next, we describe the syntax for algorithms in $\text{PCD} = (\text{G}, \text{I}, \text{P}, \text{V})$.

- $\text{PCD.G}(1^\lambda)$: sample $\text{pp}_{\text{NARK}} \leftarrow \text{NARK.G}(1^\lambda)$ and $\text{pp}_{\text{ACC}} \leftarrow \text{ACC.G}(1^\lambda)$, and output $\text{pp} := (\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}})$.
- $\text{PCD.I}(\text{pp}, \varphi)$:
 1. compute the integer $N := N(\lambda, |\varphi|, m, \sigma)$, where N is defined in Lemma 1 below.
 2. construct the circuits $R^{(0)} := R_\varphi^{(\lambda, N, k)}$ and $R^{(1)} := R_V^{(\lambda, N, k)}$, where $k := |\text{avk}(\lambda, N)| + |\text{acc.x}(\lambda, m, N)| + \sigma$.
 3. compute the index key pairs for the NARK:
 - a. $(\text{ipk}^{(0)}, \text{ivk}^{(0)}) \leftarrow \text{NARK.I}(\text{pp}_{\text{NARK}}, R^{(0)})$;
 - b. $(\text{ipk}^{(1)}, \text{ivk}^{(1)}) \leftarrow \text{NARK.I}(\text{pp}_{\text{NARK}}, R^{(1)})$.
 4. compute the index key triples for the accumulator:
 - a. $(\text{apk}^{(0)}, \text{avk}^{(0)}, \text{adk}^{(0)}) := \text{ACC.I}(\text{pp}_{\text{ACC}}, \text{pp}_\varphi = \text{pp}_{\text{NARK}}, \text{i}_\varphi = R^{(0)})$;
 - b. $(\text{apk}^{(1)}, \text{avk}^{(1)}, \text{adk}^{(1)}) := \text{ACC.I}(\text{pp}_{\text{ACC}}, \text{pp}_\varphi = \text{pp}_{\text{NARK}}, \text{i}_\varphi = R^{(1)})$.
 5. output the proving key $\text{ipk} := (\text{ipk}^{(0)}, \text{ipk}^{(1)}, \text{apk}^{(0)}, \text{apk}^{(1)})$ and verification key $\text{ivk} := (\text{ivk}^{(0)}, \text{ivk}^{(1)}, \text{avk}^{(0)}, \text{avk}^{(1)}, \text{adk})$.
- $\text{PCD.P}(\text{ipk}, z, z_{\text{loc}}, [z_i, \pi_i, \text{acc}_i]_{i=1}^m)$:
 1. sample $\pi^{(0)} \leftarrow \text{NARK.P}(\text{ipk}^{(0)}, z; z_{\text{loc}}, [z_i]_{i=1}^m)$.
 2. set predicate instance-proof pair as $(\text{qx}_0, \text{qw}_0) = ((\text{qx}_0^{(0)}, \perp), (\text{qw}_0^{(0)}, \perp))$, where $(\text{qx}_0^{(0)}, \text{qw}_0^{(0)}) := (z, \pi^{(0)}.\text{x}; \pi^{(0)}.\text{w})$.
 3. if $z_i = \perp$ for all $i \in [m]$ then set $\text{acc} := (\text{acc}^{(0)}, \text{acc}^{(1)})$, $\text{pf} := (\text{pf}^{(0)}, \text{pf}^{(1)})$, by sampling
 - a. $(\text{acc}^{(0)}, \text{pf}^{(0)}) \leftarrow \text{ACC.P}(\text{apk}^{(0)}, (\text{qx}_0, \text{qw}_0), \perp)$;
 - b. $(\text{acc}^{(1)}, \text{pf}^{(1)}) \leftarrow \text{ACC.P}(\text{apk}^{(1)}, \perp)$.
 4. if $z_i \neq \perp$ for some $i \in [m]$ then:
 - a. parse each $\pi_i = (\perp, \pi_i^{(1)})$
 - a. set predicate input instance $\text{qx}_i := (\perp, \text{qx}_i^{(1)})$ where $\text{qx}_i^{(1)} = (\text{avk}, z_i, \pi_i^{(1)}.\text{x}, \text{acc}_i.\text{x})$;
 - b. set predicate input witness $\text{qw}_i := (\perp, \text{qw}_i^{(1)})$ where $\text{qw}_i^{(1)} = (\pi_i^{(1)}.\text{w}, \text{acc}_i.\text{w})$;

- c. compute $(\text{acc}, \text{pf}) \leftarrow \text{ACC.P}(\text{apk}, [(\text{qx}_i, \text{qw}_i)]_{i=0}^m, [\text{acc}_i]_{i=1}^m)$, where $\text{apk} := (\text{apk}^{(0)}, \text{apk}^{(1)})$.
 3. compute $\pi^{(1)} \leftarrow \text{NARK.P}(\text{ipk}^{(1)}, \text{avk}, \text{acc.x}; (z, \pi_0.\text{x}), [z_i, \pi_i.\text{x}, \text{acc}_i.\text{x}]_{i=1}^m, \text{pf})$
 4. output $(\pi := (\perp, \pi^{(1)}), \text{acc})$.
- $\text{PCD.V}(\text{ivk}, (\pi, \text{acc}))$:
 - parse $\pi = (\perp, \pi^{(1)})$, $\text{acc} = (\text{acc}^{(0)}, \text{acc}^{(1)})$;
 - $b_0 \leftarrow \text{NARK.V}(\text{ivk}^{(1)}, (\text{avk}, z, \text{acc.x}), \pi^{(1)})$;
 - $b_1 \leftarrow \text{ACC.D}(\text{adk}^{(0)}, \text{acc}^{(0)})$;
 - $b_2 \leftarrow \text{ACC.D}(\text{adk}^{(1)}, \text{acc}^{(1)})$;
 - output $b_0 \wedge b_1 \wedge b_2$.

Theorem 2. $\text{PCD} = (\text{G}, \text{I}, \text{P}, \text{V})$ for a set of compliance predicates $\mathbf{F}$ with constant depth in the construction above satisfies the perfect completeness, computational knowledge soundness, and statistical zero-knowledge under the random oracle model.

We provide the proof of Theorem 2 above in the next subsection.

Lemma 1 (Efficiency [BCL⁺21]). Suppose that for every security parameter $\lambda \in \mathbb{N}$, arity m , and message size $t \in \mathbb{N}$, the ratio of accumulation verifier circuit size to index size $v^*(\lambda, m, N, k)/N$ is monotone decreasing in N . Then there exists a size function $N(\lambda, f, m, t)$ such that

$$\forall \lambda, f, m, t \in \mathbb{N}, S(\lambda, f, m, t, N(\lambda, f, m, t)) \leq N(\lambda, f, m, t).$$

4.3 Security proofs

Completeness. Assume $\mathcal{A}$ as an arbitrary adversary that breaks the completeness of PCD, i.e., causes the completeness condition given in Definition 5 to be satisfied with probability $1 - p$. We demonstrate that there exists an adversary $\mathcal{B}$, that causes the completeness condition of ACC in Definition 1 to be satisfied with probability at most $1 - p$. The constructed adversary $\mathcal{B}$ runs in the following:

1. set $\text{pp} := (\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}})$;
2. compute $(\varphi, z, z_{\text{loc}}, [z_i, \pi_i, \text{acc}_i]_{i=1}^m) \leftarrow \mathcal{A}(\text{pp})$;
3. set $(\text{apk}, \text{avk}, \text{adk}) := \text{ACC.I}((\text{pp}_{\text{ACC}}, \text{pp}_{\text{NARK}}), \varphi)$;
4. construct $[(\text{qx}_i, \text{qw}_i)]_{i=1}^m$ as in the algorithm PCD.P ;
5. output $(i_\varphi, [(\text{qx}_i, \text{qw}_i)]_{i=1}^m, [\text{acc}_i]_{i=1}^m)$, where $i_\varphi := (R_\varphi^{\lambda, N, k}, R_V^{\lambda, N, k})$.

Suppose that the adversary $\mathcal{A}$ successfully outputs $(\varphi, z, z_{\text{loc}}, [z_i, \pi_i, \text{acc}_i]_{i=1}^m)$ such that the precondition holds, i.e., (1) $\varphi \in \mathbf{F}$, (2) $\varphi(z, z_{\text{loc}}, z_1, \dots, z_m) = 1$, and (3) for all $i \in [m]$, either $z_i = \perp$ or $\text{PCD.V}(\text{ivk}, z_i, \pi_i) = 1$. Then with probability p , the completeness condition does not hold, which implies $\text{PCD.V}(\text{ivk}, z_i, \pi_i) = 0$. According to the algorithm PCD.V , at least one of the following happens:

1. $0 \leftarrow \text{NARK.V}(\text{ivk}^{(1)}, (\text{avk}, z, \text{acc.x}), \pi^{(1)});$
2. $0 \leftarrow \text{ACC.D}(\text{adk}^{(0)}, \text{acc}^{(0)});$
3. $0 \leftarrow \text{ACC.D}(\text{adk}^{(1)}, \text{acc}^{(1)}).$

Next, We discuss their probability case by case.

Case 1. If $z_i = \perp$ for all $i \in [m]$, then ensured by the perfect completeness of NARK, both instance-proof pairs $(z, \pi^{(0)}.x; \pi^{(0)}.w)$ and $(\text{avk}, z, \text{acc.x}, \pi^{(1)}.x; \pi^{(0)}.x)$ trivially holds. Therefore, items 1-3 are satisfies, which implies the existence of $i \in [m]$ such that $z_i \neq \perp$.

Case 2. If $z_i \neq \perp$ for some $i \in [m]$, we have the precondition (3): $\text{PCD.V}(\text{ivk}, z_i, \pi_i) = 1$ for all possible i . Given that Item 1 happens, i.e., $0 \leftarrow \text{NARK.V}(\text{ivk}^{(1)}, (\text{avk}, z, \text{acc.x}), \pi^{(1)}),$ we know that $R_V^{(\lambda, N, k)}$ rejects $(\text{avk}, \text{acc.x}; [z_i, \pi_i.x, \text{acc}_i.x]_{i=1}^m, \text{pf})$, and so

$$\text{ACC.V}(\text{avk}, [\text{qx}_i]_{i=0}^m, [\text{acc}_i.x]_{i=0}^m, \text{acc.x}, \text{pf}) = 0.$$

Otherwise, for item 2 and 3, we have $\text{ACC.D}(\text{adk}, \text{acc}) = 0$. Now consider the completeness experiment of ACC with adversary $\mathcal{B}$. Since $\text{pp}, \text{pp}_{\text{ACC}}$ are drawn identically to the PCD experiment, the distribution of the output of $\mathcal{A}$ is identical. Hence it holds that for all $i \in [m]$, $\Phi_\varphi(\text{pp}, R_\varphi^{(\lambda, N, k)}, \text{qx}_i^{(0)}; \text{qw}_i^{(0)}) = 1$, $\Phi_V(\text{pp}, R_V^{(\lambda, N, k)}, \text{qx}_i^{(1)}; \text{qw}_i^{(1)}) = 1$, and $\text{ACC.D}(\text{adk}, \text{acc}_i) = 1$. Particulary, for $i = 0$, by the perfect completeness of NARK, the predicate instance-proof pair must satisfy $\Phi_\varphi(\text{pp}, R_\varphi^{(\lambda, N, k)}, \text{qx}_0^{(0)}; \text{qw}_0^{(0)}) = 1$ because $\varphi(z, z_{\text{loc}}, z_1, \dots, z_m) = 1$. (precondition in Definition 1). While according to the above discussion, we have either $\text{ACC.V}(\text{avk}, [\text{qx}_i]_{i=0}^m, [\text{acc}_i.x]_{i=0}^m, \text{acc.x}, \text{pf}) = 0$ or $\text{ACC.D}(\text{adk}, \text{acc}) = 0$. As a result, the adversary $\mathcal{B}$ causes the completeness condition for the accumulation scheme ACC with probability at most $1 - p$.

Knowledge Soundness For convenience, we associate a node u with labels $(z^{(u,v)}, z_{\text{loc}}^{(u)}, \pi^{(u)}, \text{acc}^{(u)})$, where $z^{(u,v)}$ is denoted as $z^{(u)}$ if it is the unique outgoing edge of u , $\pi^{(u)}$ denotes the NARK proof, and $\text{acc}^{(u)}$ denotes the accumulator.

Now given a malicious prover P^* , we can construct an extractor $\mathcal{E}_{\text{P}^*}$ that satisfies Definition 6 through an inductive process.

Base case. In the base case, we consider the an extractor $\mathcal{E}_0$ that outputs a tree of depth-1. Trivially, we define $\mathcal{E}_0(\text{pp}_{\text{NARK}}, \text{ai})$ to compute $(\varphi, \circ, \pi, \text{acc}) \leftarrow \text{P}^*(\text{pp}_{\text{NARK}}, \text{ai})$ and output (φ, T_0) , where T_0 is a single node labeled with $(\circ, \pi, \text{acc})$.

Inductive step. Next, we inductively consider the iterative case that constructs an extractor $\mathcal{E}_j$ that outputs a tree of depth- j . We use the notation $L_{\text{T}}(j)$ to denote the vertices of T at depth j . Assume we already have an constructed extractor $\mathcal{E}_{j-1}$, an $\mathcal{E}_j$ can be constructed in the following steps.

1. Construct a malicious NARK prover $\text{NARK.P}_j^{(1)}$ as follows:

$\text{NARK.P}_j^{(1)}(\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \text{ai})$

- 1: Compute $(\varphi, \mathbf{T}_{j-1}, \text{ao}) \leftarrow \mathcal{E}_{j-1}(\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \text{ai})$.
- 2: For each vertex $v \in L_{\mathbf{T}_{j-1}}(j)$, denote its label by $(z^{(v)}, \pi^{(v)}, \text{acc}^{(v)})$.
- 3: Run the NARK indexer $(\text{ipk}^{(1)}, \text{ivk}^{(1)}) := \text{NARK.I}(\text{pp}_{\text{NARK}}, R^{(1)})$.
- 4: Run the accumulator indexer $(\text{apk}^{(0)}, \text{avk}^{(0)}, \text{adk}^{(0)}) := \text{ACC.I}(\text{pp}, R^{(0)})$.
- 5: Run the accumulator indexer $(\text{apk}^{(1)}, \text{avk}^{(1)}, \text{adk}^{(1)}) := \text{ACC.I}(\text{pp}, R^{(1)})$.
- 6: Output $(\mathbf{i}, \mathbf{x}, \boldsymbol{\pi}, \text{ao}') :=$
 $\left(R^{(1)}, ((\text{avk}^{(0)}, \text{avk}^{(1)}), z^{(v)}, \text{acc}^{(v)})_{v \in L_{\mathbf{T}_{j-1}}(j)}, (\pi^{(v,1)})_{v \in L_{\mathbf{T}_{j-1}}(j)}, (\varphi, \mathbf{T}_{j-1}, \text{ao}) \right)$

2. Let $\mathcal{E}_{\text{NARK.P}_j^{(1)}}$ be the extractor that corresponds to $\text{NARK.P}_j^{(1)}$ in terms of $\text{ipk}^{(0)}$, via the knowledge soundness of the non-interactive argument NARK.
3. Construct a malicious accumulation scheme prover ACC.P_j^* as follows:

$\text{ACC.P}_j^*(\text{pp}_{\text{ACC}}, \text{pp}_{\text{NARK}}, \text{ai})$

- 1: Run the extractor $(\mathbf{i}, \mathbf{x}, \mathbf{w}, \text{ao}') \rightarrow \mathcal{E}_{\text{NARK.P}_j^*}(\text{pp}, \text{pp}_{\text{ACC}}, \text{ai})$.
- 2: Parse the auxiliary output ao' as $(\varphi, \mathbf{T}', \text{ao})$.
- 3: For each vertex $v \in L_{\mathbf{T}_{j-1}}(j)$,
 obtain $\text{acc}^{(v)}$ from $\mathbf{T}'$;
 obtain $[z_i^{(v)}, \pi_i^{(v)}.x, \text{acc}_i^{(v)}.x]_{i=0}^m$ and $\text{pf}^{(v)}$ from $\mathbf{w}^{(v)} \in \mathbf{w}$;
 let $S_j := \{v \in L_{\mathbf{T}_{j-1}}(j) : \exists i \in [m], z_i^{(v)} \neq \perp\}$;
 attach m children to each $v \in S_j$, where the i -th child is labeled with $z_i^{(v)}$;
 parse $\pi_i^{(v)}$ as $(\perp, \pi_i^{(v,1)})$ for $i \in [m]$
 define $\text{qx}_i^{(v,1)} := ((\text{avk}, z_i^{(v)}, \text{acc}_i^{(v)}.x), \pi_i^{(v,1)}.x)$ $\text{qx}_i^{(v)} := (\perp, \text{qx}_i^{(v,1)})$ for $i \in [m]$
 define $\text{qx}_i^{(v)} := (\perp, \text{qx}_i^{(v,1)})$ for $i \in [m]$
 parse $\pi_0^{(v)}$ as $(\pi_0^{(v,0)}, \perp)$ for $i \in [m]$
 define $\text{qx}_0^{(v,0)} := (z_i^{(v)}, \pi_0^{(v,0)})$
 define $\text{qx}_0^{(v)} := (\text{qx}_0^{(v,0)}, \perp)$
- 4: Output $\left((\mathbf{i}^{(v)}, \text{acc}^{(v)}, \text{pf}^{(v)}, [\text{qx}_i^{(v)}]_{i=1}^m, [\text{acc}_i^{(v)}.x]_{i=1}^m)_{v \in S_j}, (\varphi, \mathbf{T}', \text{ao}) \right)$.

4. Let $\mathcal{E}_{\text{ACC.P}_j^*}$ be the extractor that corresponds to ACC.P_j^* , via the knowledge soundness of the accumulation scheme ACC.
5. Construct a malicious NARK prover $\text{NARK.P}_j^{(0)}$ as follows:

$\text{NARK.P}_j^{(1)}(\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \text{ai})$

- 1: Run the extractor $\left((i^{(v)}, \text{acc}^{(v)}, \text{pf}^{(v)}, [\text{qx}_i^{(v)}, \text{qw}_i^{(v)}]_{i=0}^m, [\text{acc}_i^{(v)}]_{i=1}^m)_{v \in S_j}, \text{ao}' \right) \leftarrow \mathcal{E}_{\text{ACC.P}_j^*}(\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \text{ai})$.
- 2: For $i = 0$, parse $(\text{qx}_0^{(v)}, \text{qw}_0^{(v)})$ as $(\text{qx}_0^{(v,0)}, \text{qw}_0^{(v,0)}) := ((z, \pi_0^{(v,0)}).\text{x}), \pi_0^{(v,0)}.\text{w})$.
- 3: Run the NARK indexer $(\text{ipk}^{(0)}, \text{ivk}^{(0)}) := \text{NARK.I}(\text{pp}_{\text{NARK}}, R^{(0)})$.
- 4: Output $(\mathbf{i}, \mathbf{x}, \boldsymbol{\pi}, \text{ao}') :=$

$$\left(R^{(0)}, z^{(v)}, (\pi^{(v,0)})_{v \in L_{T_{j-1}}(j)}, (\varphi, T_{j-1}, \text{ao}) \right)$$

6. Let $\mathcal{E}_{\text{NARK.P}_j^{(0)}}$ be the extractor that corresponds to $\text{NARK.P}_j^{(0)}$ in terms of $\text{ipk}^{(1)}$, via the knowledge soundness of the non-interactive argument NARK.
7. Define the extractor $\mathcal{E}_j$ as follows:

$\mathcal{E}_j(\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \text{ai})$

- 1: Run the extractor $\left((i^{(v)}, \text{acc}^{(v)}, \text{pf}^{(v)}, [\text{qx}_i^{(v)}, \text{qw}_i^{(v)}]_{i=0}^m, [\text{acc}_i^{(v)}]_{i=1}^m)_{v \in S_j}, \text{ao}' \right) \leftarrow \mathcal{E}_{\text{ACC.P}_j^*}(\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \text{ai})$.
- 2: Parse the auxiliary output ao' as (φ, T', ao) . 3: Let $S_j := \{v \in L_{T'}(j) : \exists i \in [m], z_i^{(v)} \neq \perp\}$;
- 4: Run the extractor $(\mathbf{i}, \mathbf{x}, \mathbf{w}, \text{ao}') \leftarrow \mathcal{E}_{\text{NARK.P}_j^{(0)}}(\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \text{ai})$.
- 5: Obtain the local data $z_{\text{loc}}^{(b)}, [z_i^{(v)}]_{i=1}^m$ from $\mathbf{w}^{(v)} \in \mathbf{w}$
- 6: Parse each $\text{qx}_i^{(v)}$ as $((\text{avk}^{(v)}, z_i^{(v)}, \text{acc}_i^{(v)}).\text{x}), \pi_i^{(v)}.\text{x})$ and $\text{qw}_i^{(v)}$ as $\pi_i^{(v)}.\text{w}$; combine each pair $(\pi_i^{(v)}.\text{x}, \pi_i^{(v)}.\text{w})$ into a proof $\pi_i^{(v)}$.
- 7: Append $z_{\text{loc}}^{(v)}$ to the label of v in T' and compare $[z_i^{(v)}]_{i=1}^m$.
- 8: Construct T_j from T' by adding, for each vertex $v \in S_j$, $(\pi_i^{(v)}, \text{acc}_i^{(v)})$ to the label of its i -th child.
- 9: Output $(\varphi, T_j, \text{ao})$.

Given that the extractor for NARK and ACC runs in expected polynomial time, the extractor $\mathcal{E}_{\text{PCD.P}} = \mathcal{E}_{d(\varphi)}$ also runs in expected polynomial time. Given that the knowledge soundness holds for NARK and ACC, the extracted transcript tree is F-compliant. More details can be referred to [BCL⁺21].

Zero Knowledge The simulator Sim operates as follows.

$\text{Sim}(1^\lambda)$:

- 1: Sample parameters for NARK: $\text{pp}_{\text{NARK}} \leftarrow \text{NARK.G}(1^\lambda)$.
- 2: Sample simulated parameters for the accumulation scheme: $(\text{pp}_{\text{ACC}}, \tau_{\text{ACC}}) \leftarrow \text{ACC.Sim}(1^\lambda)$.
- 3: Output $(\text{pp} := (\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}), (\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \tau_{\text{ACC}}))$.

$\text{Sim}((\text{pp}_{\text{NARK}}, \text{pp}_{\text{ACC}}, \tau_{\text{NARK}}, \tau_{\text{ACC}}), \varphi, z):$
1: Compute accumulator keys: $(\text{apk}^{(0)}, \text{avk}^{(0)}, \text{adk}^{(0)}) := \text{ACC.I}(\text{pp}_{\text{ACC}}, \text{pp}_{\varphi} := \text{pp}_{\text{NARK}}, i_{\varphi} = R^{(0)})$.
2: Compute accumulator keys: $(\text{apk}^{(1)}, \text{avk}^{(1)}, \text{adk}^{(1)}) := \text{ACC.I}(\text{pp}_{\text{ACC}}, \text{pp}_{\varphi} := \text{pp}_{\text{NARK}}, i_{\varphi} = R^{(1)})$.
3: Sample instances $[\text{qx}_i]_{i=0}^n, [\text{acc}_j]_{j=1}^m$.
4: Sample simulated accumulator and accumulation proof: $(\text{acc}, \text{pf}) \leftarrow \text{ACC.Sim}(\tau_{\text{ACC}}, \text{pp}_{\varphi}, i_{\varphi}, [\text{qx}_i]_{i=0}^n, [\text{acc}_j]_{j=1}^m)$
5: Compute NARK proof: $\pi^{(1)} \leftarrow \text{NARK.P}(\text{ipk}^{(1)}, \text{avk}, \text{acc.x}; (z, \pi_0.x), [z_i, \pi_i.x, \text{acc}_i.x]_{i=1}^m, \text{pf})$
6: Output $(\pi := (\perp, \pi^{(1)}), \text{acc})$.

Then we argue that the simulated (π, acc) is indistinguishable. Given that the accumulator scheme is zero-knowledge, it is clear that acc is indistinguishable from the accumulator output by the honest accumulator prover acc.P with inputs $(\text{apk}, [(\text{qx}_i, \text{qw}_i)]_{i=0}^n, [\text{acc}_j]_{j=1}^m)$. To argue the indistinguishability of π is kind of trivial since the NARK does not provide zero-knowledge. Nevertheless, we note that π is generated by the NARK prover with the instance-witness pair as follows:

$$\mathbf{x} := (\text{ipk}^{(1)}, \text{avk}, \text{acc.x}), \mathbf{w} := ((z, \pi_0.x), [z_i, \pi_i.x, \text{acc}_i.x]_{i=1}^m, \text{pf})$$

where $(z, \pi.x)$ can be parsed from qx_0 , $z_i, \pi_i.x$ can be parsed from qx_i for $i \in [n]$. The accumulation proof can be sampled from ACC.Sim . Therefore, the simulated π is also indistinguishable from the proof honestly generated by a NARK with inputs instance-witness pair simulated by ACC.Sim .

5 Concrete Instantiations

In this section, we present an efficient zero-knowledge accumulation scheme, providing a concrete instantiation of zero-knowledge PCD as established in Theorem 1. We begin by reviewing the necessary definitions in Section 5.1. Subsequently, we introduce a zero-knowledge accumulation scheme for special-sound protocols, which is constructed based on a variant of Protogalaxy [EG23]. Finally, we present a formal security proof for the proposed scheme, which also serves as a complement to the analysis in [EG23].

5.1 Basic Definitions

Special-Sound Protocols. We adopt the definitions of special-sound protocols [AC20] given in [BC23, EG23]. Define a $(2\mu - 1)$ -move special-sound protocol Π_{sps} for relation R according to Definition 14 in Appendix, with verifier degree d , output length n and two PPT algorithms $P_{\text{sps}}, V_{\text{sps}}$. Let public parameters pp contain μ, d, n , public input instance be $\mathbf{x} = \mathbf{x}$, private input witness be

$\mathbf{w} = \mathbf{w}$. At each round $i \in [\mu]$, the prover runs P_{sps} to compute the next message $\mathbf{m}_i \in \mathbb{F}^\mu$ and sends to the verifier, which is based on previous information including instance vector $\mathbf{x}$, witness vector $\mathbf{w}$ and transcripts $\{\mathbf{m}_j, r_j\}_{j \in [i]}$ in previous i rounds. After receiving the i -th round message from the prover, the verifier randomly samples a challenge r_i as a response. Then the two parties repeat the above procedure until $i = \mu$. After receiving the final message $\mathbf{m}_\mu$, the verifier runs the algorithm V_{sps} for checking the validity of the witness $\mathbf{w}$. Note that V_{sps} is an algebraic map with maximum degree d as below, where $f_k^{\text{V}_{\text{sps}}}$ is a homogeneous algebraic map with degree k :

$$V_{\text{sps}}(\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}) := \sum_{k=0}^d f_k^{\text{V}_{\text{sps}}}(\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}) = \mathbf{0}^n, \quad (2)$$

where degree- k homogeneity says that each monomial term of $f_j^{\text{V}_{\text{sps}}}$ has degree exactly k . We denote the $(2\mu - 1)$ -move protocol as Π_{sps} and provide the step-by-step description below.

Special-sound protocol Π_{sps} for relation $\mathcal{R}$

P's inputs: $\text{pp}_{\text{sps}}, \mathbf{x}; \mathbf{w}$

V's inputs: $\text{pp}_{\text{sps}}, \mathbf{x}$

- 1: For $i \in [\mu - 1]$
 - $\mathbf{m}_i \leftarrow P_{\text{sps}}(\mathbf{x}, \mathbf{w}, \{\mathbf{m}_j, r_j\}_{j=1}^{i-1})$
 - $P \rightarrow V : \mathbf{m}_i$
 - $V \rightarrow P : r_i \leftarrow \$_\mathbb{F}$
- 2: P computes $\mathbf{m}_\mu \leftarrow P_{\text{sps}}(\mathbf{x}, \mathbf{w}, \{\mathbf{m}_j, r_j\}_{j=1}^{\mu-1})$
- 3: $P \rightarrow V : \mathbf{m}_\mu$
- 4: V checks if $V_{\text{sps}}(\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}) = \mathbf{0}^n$

Zero-Knowledge Sumcheck Protocols. The sumcheck protocol is an interactive proof proposed by Lund et al. [LFKN92]. Assume $\tilde{f}(\mathbf{X})$ as an m -variate polynomial with the maximum individual degree of d . The prover aims to convince the verifier that $s = \sum_{\mathbf{x} \in \{0,1\}^m} \tilde{f}(\mathbf{x})$. We denote the $(2m + 1)$ -move sum-check protocol as Π_{sc} and provide the step-by-step description in Π_{sc} . We mention that the interactive sum-check protocol satisfies both completeness and soundness according to [LFKN92], Where the soundness ensures that any malicious prover can not forge a valid proof of a wrong statement with probability at most $d \cdot m / |\mathbb{F}|$ by the Schwartz-Zippel lemma in Lemma 2.

To further achieve the zero-knowledge property, Chiesa et al. proposed an approach by masking the polynomial f in the sumcheck protocol by a random polynomial g in [CFS17], where the random polynomial g has the same variables and individual degrees of f . Xie et al. in [XZZ⁺19] reduce the size of the random

polynomial to $O(d \cdot m)$. Briefly, the random polynomial can be constructed as

$$g(\mathbf{X}) = r_0 + r_1(X_1) + r_2(X_2) + \cdots + r_m(X_m), \quad (3)$$

where $r_i(x_i) = r_{i,1}x_i + r_{i,2}x_i^2 + \cdots + r_{i,d}x_i^d, i \in [m]$ is a random univariate polynomial of degree d . We adopt the results in previous works [CFS17,XZZ⁺19] that state that a simulator that forges an indistinguishable transcript can be constructed.

Sum-check protocol Π_{sc}
$\mathbf{P}_{\text{sc}}$'s inputs: $\mathbf{pp}, s; \tilde{f}(\mathbf{X})$ $\mathbf{V}_{\text{sc}}$'s inputs: $\mathbf{pp}, s$ 1: For $i \in [m]$ $\mathbf{P}_{\text{sc}}$ computes $f_i(X) = \sum_{x_{i+1}, \dots, x_n \in B_{n-i}} \tilde{f}(r_1, \dots, r_{i-1}, X, x_{i+1}, \dots, x_n)$ $\mathbf{P}_{\text{sc}} \rightarrow \mathbf{V}_{\text{sc}} : f_i(X)$ for $i = 1$, $\mathbf{V}_{\text{sc}}$ checks if $s + \gamma \cdot r = f_1(0) + f_1(1)$ else, $\mathbf{V}_{\text{sc}}$ checks if $f_{i-1}(r_{i-1}) = f_i(0) + f_i(1)$ 2: $\mathbf{V}_{\text{sc}}$ checks if $f_m(r_m) = \tilde{f}(r_1, \dots, r_m)$.

Lemma 2 (Schwartz-Zippel lemma). *Assume an m -variate non-zero polynomial $\tilde{f}(\mathbf{X}) \in \mathbb{F}^{(\leq d)}[\mathbf{X}]$, $\mathbf{X} \in \mathbb{F}^m$, then it holds that*

$$\Pr_{\mathbf{r} \leftarrow \mathbb{F}^m} [\tilde{f}(\mathbf{r}) = 0] \leq \frac{d \cdot m}{|\mathbb{F}|} \quad (4)$$

Equality Polynomials. Let $\tilde{f}(\cdot) : \mathbb{F}^n \rightarrow \mathbb{F}$ be a *multivariate polynomial* with n input elements over $\mathbb{F}$, its total degree $\deg(\tilde{f}) \in \mathbb{N}$ is defined as the maximum degree over all monomials. A multivariate polynomial is a *multilinear* polynomial if the degree of each variable is at most one. Next, we denote the multilinear extension of a vector $\mathbf{f}$ as a unique multilinear n -variate polynomial $\tilde{f}(\cdot) : \mathbb{F}^n \rightarrow \mathbb{F}$ such that $\tilde{f}(\text{Bits}(i)) = \mathbf{f}[i]$ for all $i \in [n]$, where the binary representation $\text{Bits}(i)$ belongs to the boolean hypercube $B_{\log n} = \{0, 1\}^{\log n}$. The multilinear extension can be computed as $\tilde{f}(\mathbf{X}) = \sum_{i \in [n]} \mathbf{f}[i] \cdot \tilde{eq}(\mathbf{X}, \text{Bits}(i))$, where $eq(\mathbf{X}, \text{Bits}(i))$ is an extension of the equality function $eq(\cdot)$. For simplicity, we sometimes use the integer index instead of a binary vector to represent the equality function as $eq_i(\mathbf{X}) := \tilde{eq}(\mathbf{X}, \text{Bits}(i)) = \prod_{j=0}^{\log n - 1} (\text{Bits}(i)[j] \cdot X_j + (1 - \text{Bits}(i)[j])(1 - X_j))$, $i \in [n]$. Denote $\mathbb{F}^{(< d)}[\mathbf{X}]$ where $\mathbf{X} \in \mathbb{F}^n$ as $\mathcal{F}_n^{(< d)}$ for short.

To reduce the potential cross terms generated in accumulating predicate pairs and accumulators, Eagen et al. introduced an elegant linear combination technique based on Lagrange bases in [EG23], which is extended to equality polynomials in [ZGGX25] as the following corollary:

Corollary 1. *A multilinear equality polynomial $eq_i(\mathbf{X})$ defined over the Boolean hypercube $\{0, 1\}_{\log m}$ satisfies the following facts:*

- $(eq_i(\mathbf{b}))^d = eq_i(\mathbf{b})$ holds for all $d \in \mathbb{N}$ and $\mathbf{b} \in \{0, 1\}_{\log m}$,
- $eq_i(\mathbf{b}) \cdot eq_j(\mathbf{b}) = 0$ holds for all $\mathbf{b} \in \{0, 1\}_{\log m}$ if and only if $i \neq j \in [\ell]$.

The proof idea can be referred to [ZGGX25]. The above corollary inspires us to conduct an accumulation without introducing cross terms, which is especially important in accumulating multiple high-degree instances. For example, when computing the accumulation of two equations $V(\boldsymbol{\alpha}) = v_\alpha$ and $V(\boldsymbol{\beta}) = v_\beta$, the derived equation holds for all $b \in \{0, 1\}$:

$$V(\boldsymbol{\alpha}) \cdot eq_0(b) + V(\boldsymbol{\beta}) \cdot eq_1(b) = v_\alpha \cdot eq_0(b) + v_\beta \cdot eq_1(b).$$

Therefore, the prover only needs to provide an additional zero-check proof.

We mention another common technique [XZZ⁺19, Set20] used in our construction, which can “replace” the random point of an evaluation claim with a new one. That is, given a claim $\tilde{f}(\mathbf{r}_x) = v$, we can derive a new claim $\tilde{f}(\mathbf{r}'_x) = v'$ by running a sum-check protocol for the target polynomial $\tilde{G}(\mathbf{x}) = \tilde{eq}(\mathbf{r}_x, \mathbf{x}) \cdot \tilde{f}(\mathbf{x})$ with randomness $\mathbf{r}'_x$.

5.2 Full Scheme

Let $\rho_{\text{ACC}}, \rho_{\text{NARK}}, H$ be three random oracles. Next, we describe a zero-knowledge accumulation scheme for the verification algorithm V_{NARK} of $\text{NARK} := \text{FS}[\Pi_{\text{sps}}^{\text{cm}}]$, where the NARK is transformed from the special sound protocols Π_{sps} by committing all messages and applying the Fiat-Shamir heuristic. For simplicity, we assume the commitment scheme provides a hiding property, where each commitment consists of a random element implicitly. The construction of the accumulation scheme generally follows the framework in [BC23, EG23], with the differences compared to the non-zero-knowledge version highlighted in blue texts.

Predicate. The predicate $\Phi := V_{\text{NARK}}$ to be accumulated is the verification algorithm of the NARK system $\text{FS}[\Pi_{\text{sps}}^{\text{cm}}]$ for relation $\mathcal{R}$. Namely, the predicate takes inputs as public parameters pp_{sps} , commitment key ck , index $i_\Phi := V_{\text{sps}}$, instance vector $\mathbf{x} \in \mathbb{F}^k$, random challenges $[r_i]_{i=1}^\mu$, and a NARK proof $\pi = (\pi.x, \pi.w)$, where $\pi.x = [C_i]_{i=1}^\mu$, $\pi.w = [\mathbf{m}_i]_{i=1}^\mu$, $C_i = \text{Commit}(\text{ck}, \mathbf{m}_i), \forall i \in [\mu]$. Denote qx, qw as

$$\text{qx} := (\mathbf{x}, [C_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}), \quad \text{qw} := ([\mathbf{m}_i]_{i=1}^\mu).$$

Hence, the predicate can be checked by accessing a random oracle ρ_{NARK} as follows:

- $r_i = \rho_{\text{NARK}}(r_{i-1}, C_i)$ for all $i \in [\mu - 1]$ ($r_0 = \rho_{\text{NARK}}(\text{pi})$);
- $C_i = \text{Commit}(\text{ck}, \mathbf{m}_i)$ for all $i \in [\mu]$;
- $\sum_{k=0}^d f_k^{\text{V}_{\text{sps}}}(\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}) = \mathbf{0}^n$,

Here $f_k^{\text{V}_{\text{sps}}}$ is a degree- k homogeneous algebraic map that outputs n field elements. In this paper, we do not consider the non-uniform case for simplicity, i.e., accumulating instances with different indexes i_Φ . Therefore, the function V_{sps} only inputs $\mathbf{x}$ as part of pi and leaves the public parameters and index as invariants.

Accumulator. The accumulator has the following format:

- Accumulator instance $\text{acc.x} := (h, [C_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}, E, \beta, \mathbf{v}_g)$, where $\mathbf{x}$ is compressed into h by a random oracle H , $E \in \mathcal{C}$ is the accumulated commitment to the error terms, $\beta \in \mathbb{F}^{\log m+1}$ is challenge, and $\mathbf{v}_g \in \mathbb{F}^n$ is a vector of evaluations.
- Accumulator witness $\text{acc.w} := (\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, \mathbf{G}(\mathbf{X}))$, where $\mathbf{x}$ is the accumulated NARK prover instance vector, $[\mathbf{m}_i]_{i=1}^\mu$ are the accumulated NARK prover messages, and $\mathbf{G}(\mathbf{X}) \in \mathbb{F}^{(\leq d)}[\mathbf{X}]$ is vector of random polynomials, which satisfy:

$$\sum_{k=0}^d f_k^{\text{V}_{\text{sps}}}(\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}) = \mathbf{e} \wedge E = \text{Commit}(\text{ck}, \mathbf{e}) \wedge \mathbf{v}_g = \mathbf{G}(\beta)$$

Generator. The accumulation generator ACC.G receives as input the security parameter 1^λ and outputs pp .

Indexer. The accumulation indexer ACC.I receives as inputs accumulation parameters pp and predicate parameters pp_Φ , commitment key ck , and a predicate index i_Φ . The indexer outputs

- accumulator proving key $\text{apk} := (\text{pp}, \text{pp}_\Phi, \text{ck}, i_\Phi)$;
- accumulator verification key $\text{avk} := (\text{pp}, i_\Phi)$;
- decision key $\text{adk} := (\text{pp}, \text{ck})$.

Accumulation prover. On input accumulator proving key apk , accumulators $[\text{acc}_j]_{j=1}^m$, predicate instance-proof pairs $[(\mathbf{qx}_j, \mathbf{qw}_j)]_{j=1}^m$ as defined above, the prover ACC.P accumulates them into a new accumulator acc by running the following process:

1. For $j \in [m]$:
 - check the consistency of challenges: $\mathbf{qx}_j.r_i = \rho_{\text{NARK}}(\mathbf{qx}_j.r_{i-1}, \mathbf{qx}_j.C_i)$ for all $i \in [\mu - 1]$ ($\mathbf{qx}_j.r_1 = \rho_{\text{NARK}}(\mathbf{qx}_j.\mathbf{x})$);
 - check the consistency of hash value $\text{acc}_j.x.h = H(\text{acc}_j.w.\mathbf{x})$;
2. Sample a random predicate instance-witness pair $\mathbf{qx}_0, \mathbf{qw}_0$ with $\mathbf{qx}_0 \leftarrow \mathbb{F}^k$ and $\mathbf{qw}_0 \leftarrow \mathbb{F}^t \forall i \in [\mu]$, where $\sum_{k=0}^d f_k^{\text{V}_{\text{sps}}}(\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}) = \mathbf{e}_0$.

3. Compute a vector of $(\log m + 1)$ -variate polynomials as:

$$\begin{aligned} \mathbf{F}(\mathbf{X}) &:= \sum_{k=0}^d f_k^{\text{Vsp}}(\mathbf{x}(\mathbf{X}), [\mathbf{m}_i(\mathbf{X})]_{i=1}^\mu, [r_i(\mathbf{X})]_{i=1}^{\mu-1}) \in (\mathbb{F}^{(\leq d)}[\mathbf{X}])^n \\ \mathbf{x}(\mathbf{X}) &= eq_0(\mathbf{X}) \cdot \mathbf{qx}_0 \cdot \mathbf{x} + \sum_{j=1}^m [eq_{j-1}(\mathbf{X}) \cdot \mathbf{qx}_j \cdot \mathbf{x} + eq_{j+m-1}(\mathbf{X}) \cdot \mathbf{acc}_j \cdot \mathbf{w} \cdot \mathbf{x}], \\ \mathbf{m}_i(\mathbf{X}) &= eq_0(\mathbf{X}) \cdot \mathbf{qw}_0 \cdot \mathbf{m}_i + \sum_{j=1}^m [eq_{j-1}(\mathbf{X}) \cdot \mathbf{qw}_j \cdot \mathbf{m}_i + eq_{j+m-1}(\mathbf{X}) \cdot \mathbf{acc}_j \cdot \mathbf{w} \cdot \mathbf{m}_i], \\ r_i(\mathbf{X}) &= eq_0(\mathbf{X}) \cdot \mathbf{qx}_0 \cdot r_i + \sum_{j=1}^m [eq_{j-1}(\mathbf{X}) \cdot \mathbf{qx}_j \cdot r_i + eq_{j+m-1}(\mathbf{X}) \cdot \mathbf{acc}_j \cdot \mathbf{x} \cdot r_i], \end{aligned}$$

where $\mathbf{F}(\mathbf{b}) = eq_0(\mathbf{b}) \cdot \mathbf{e}_0 + \sum_{j=1}^m eq_{j+m-1}(\mathbf{b}) \cdot \mathbf{acc}_j \cdot \mathbf{w} \cdot \mathbf{e} = \mathbf{e}(\mathbf{b})$ for all $\mathbf{b} \in \{0, 1\}^{\log m + 1}$.

3. $h \leftarrow H(\mathbf{x})$.
4. $\alpha \in \mathbb{F}^{\log m + 1}, \gamma \in \mathbb{F} \leftarrow \rho_{\text{ACC}}([\mathbf{acc}_j \cdot \mathbf{x}]_{j=1}^m, [\mathbf{qx}_j]_{j=1}^m, h)$
5. Run non-interactive sum-check protocols $\text{FS}[\Pi_{\text{sc}}]$ in parallel for the following $n + m \cdot n$ statements

$$\begin{aligned} \sum_{\mathbf{b} \in \{0, 1\}^{\log m + 1}} [eq(\mathbf{b}, \alpha) \cdot \mathbf{F}(\mathbf{b}) + \gamma \cdot \mathbf{G}'(\mathbf{b})] &= \sum_{\mathbf{b} \in \{0, 1\}^{\log m + 1}} eq(\mathbf{b}, \alpha) \cdot \mathbf{e}(\mathbf{b}) + \gamma \cdot \mathbf{e}'_g \in \mathbb{F}^n. \\ \sum_{\mathbf{b} \in \{0, 1\}^{\log m + 1}} eq(\mathbf{acc}_j \cdot \mathbf{x}, \beta, \mathbf{b}) \cdot \mathbf{acc}_j \cdot \mathbf{w} \cdot \mathbf{G}(\mathbf{b}) &= \mathbf{acc}_j \cdot \mathbf{x} \cdot \mathbf{v}_g \forall j \in [m] \end{aligned}$$

where $\mathbf{G}'(\mathbf{b}) \in (\mathbb{F}^{(\leq d)}[\mathbf{X}])^n$ is a vector of random polynomials and $\mathbf{e}'_g := \sum_{\mathbf{b} \in \{0, 1\}^{\log m + 1}} \mathbf{G}'(\mathbf{b})$ according to Equation 3. The obtained transcript is denoted as:

$$\text{tr} := (\mathbf{e}'_g, [\mathbf{F}_j(X) \in (\mathbb{F}^{(\leq d)}[\mathbf{X}])^{n+m \cdot n}]_{j=1}^{\log m + 1}, \beta, \gamma, \mathbf{v} \in \mathbb{F}^n, [\mathbf{v}'_{g,j}]_{j=1}^m).$$

where $\mathbf{F}_j(X)$ is a $n+m \cdot n$ -length vector of polynomials in round j , $\mathbf{v}'_{g,j} := \mathbf{acc}_j \cdot \mathbf{w} \cdot \mathbf{G}(\beta)$.

6. Compute $\mathbf{v}'_g := \mathbf{G}'(\beta) \in \mathbb{F}^n$, and set

$$\mathbf{e} := \mathbf{F}(\beta) = (\mathbf{v} - \gamma \cdot \mathbf{v}'_g) \cdot eq^{-1}(\beta, \alpha) \quad (5)$$

7. Compute $E \leftarrow \text{Commit}(\text{ck}, \mathbf{e}) \in \mathcal{C}$.

8. Set vectors

$$\begin{aligned} \mathbf{v}_0 &= ([\mathbf{qx}_0 \cdot r_i]_{i=1}^{\mu-1}, [\mathbf{qx}_0 \cdot C_i]_{i=1}^\mu, \mathbf{v}'_g, \mathbf{qw}_0 \cdot \mathbf{x}, [\mathbf{qw}_0 \cdot \mathbf{m}_i]_{i=1}^\mu, \mathbf{G}'(\mathbf{X})) \\ \mathbf{v}_j &= ([\mathbf{qx}_j \cdot r_i]_{i=1}^{\mu-1}, [\mathbf{qx}_j \cdot C_i]_{i=1}^\mu, \perp, \mathbf{qw}_j \cdot \mathbf{x}, [\mathbf{qw}_j \cdot \mathbf{m}_i]_{i=1}^\mu, \perp), j \in [1, m] \\ \mathbf{v}_{j+m} &= \left([\mathbf{acc}_j \cdot \mathbf{x} \cdot r_i]_{i=1}^{\mu-1}, [\mathbf{acc}_j \cdot \mathbf{x} \cdot C_i]_{i=1}^\mu, \mathbf{v}'_{g,j}, \right. \\ &\quad \left. \mathbf{acc}_j \cdot \mathbf{w} \cdot \mathbf{x}, [\mathbf{acc}_j \cdot \mathbf{w} \cdot \mathbf{m}_i]_{i=1}^\mu, \mathbf{acc}_j \cdot \mathbf{w} \cdot \mathbf{G}(\mathbf{X}) \right), j \in [1, m]. \end{aligned}$$

8. Compute the accumulated vector

$$\mathbf{v} := ([r_i]_{i=1}^{\mu-1}, [C_i]_{i=1}^\mu, \mathbf{v}_g, \mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, \mathbf{G}) = \sum_{j=0}^{2m} eq_{j-1}(\beta) \cdot \mathbf{v}_j.$$

9. Set the new accumulator acc as

$$\begin{aligned}\text{acc.x} &:= (\text{pp}_\Phi, \text{i}_\Phi, h, [C_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}, E, \beta, \mathbf{v}_g) \\ \text{acc.w} &:= (\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, \mathbf{G}(\mathbf{X})).\end{aligned}$$

10. set accumulation proof $\text{pf} := (\mathbf{qx}_0, \text{tr}, \mathbf{v}'_g)$.

Accumulation verifier. On input accumulator verification key avk , accumulator instances $[\text{acc}_j.\mathbf{x}]_{j=1}^m$, predicate instances $[\mathbf{qx}_j]_{j=1}^m$, a new accumulator instance acc.x , and an accumulation proof pf , the verifier ACC.V checks the accumulation by running the following process:

1. For $j \in [0, m]$:
 - check the consistency of challenges: $\mathbf{qx}_j.r_i = \rho_{\text{NARK}}(\mathbf{qx}_j.r_{i-1}, \mathbf{qx}_j.C_i)$ for all $i \in [\mu - 1]$ ($\mathbf{qx}_j.r_1 = \rho_{\text{NARK}}(\mathbf{qx}_j.\mathbf{x})$);
 - check the consistency of hash value $\text{acc}_j.\mathbf{x}.h = H(\text{acc}_j.\mathbf{w}.\mathbf{x})$;
2. Check the correctness of sum-check proof pf.tr by running the verifier of $\text{FS}[\Pi_{\text{sc}}]$ in terms of $\beta, \gamma \leftarrow \rho_{\text{ACC}}([\text{acc}_j.\mathbf{x}]_{j=1}^m, [\mathbf{qx}_j]_{j=1}^m, h)$.
3. Set vectors

$$\begin{aligned}\mathbf{v}_0 &= ([\mathbf{qx}_0.r_i]_{i=1}^{\mu-1}, [\mathbf{qx}_0.C_i]_{i=1}^\mu, \perp, \mathbf{v}'_g) \\ \mathbf{v}_j &= ([\mathbf{qx}_j.r_i]_{i=1}^{\mu-1}, [\mathbf{qx}_j.C_i]_{i=1}^\mu, \perp, \perp), j \in [1, m] \\ \mathbf{v}_{j+m} &= ([\text{acc}_j.\mathbf{x}.r_i]_{i=1}^{\mu-1}, [\text{acc}_j.\mathbf{x}.C_i]_{i=1}^\mu, \text{acc}_j.\mathbf{x}.E, \mathbf{v}'_{g,j}), j \in [1, m].\end{aligned}$$

4. Check if $\text{acc.x} \cdot ([r_i]_{i=1}^{\mu-1}, [C_i]_{i=1}^\mu, E, \mathbf{v}_g) = \sum_{j=0}^{2m} eq_{j-1}(\beta) \cdot \mathbf{v}_j$.

Decider. On inputs the decider key adk and an accumulator

$$\text{acc} = (\text{acc.x} = (h, [C_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}, E, \beta, \mathbf{v}_g), \text{acc.w} = (\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, \mathbf{G}(\mathbf{X})),$$

the decider does the checks described as follows.

1. check $C_i \stackrel{?}{=} \text{Commit}(\text{ck}, \mathbf{m}_i)$ for all $i \in [\mu]$;
2. compute $\mathbf{e} \leftarrow \sum_{k=0}^d f_k^{\text{V}_{\text{sps}}}(\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1})$;
3. check if $E = \text{Commit}(\text{ck}, \mathbf{e})$,
4. check if $\mathbf{G}(\beta) = \mathbf{v}_g$.

Theorem 3. Let V_{NARK} be the verifier of the NARK system $\text{FS}[\Pi_{\text{sps}}^{\text{cm}}]$ in terms of ρ_{sps} for relation R^n under the random oracle model. Let $\text{CM} = (\text{Setup}, \text{Commit}, \text{Open}, \text{Verify})$ be a homomorphic commitment scheme with binding and hiding properties. Let ρ_{acc}, H be another two random oracles. The accumulation scheme $\text{ACC} = (\text{G}, \text{I}, \text{P}, \text{V}, \text{D})$ for V_{NARK} satisfies perfect completeness, knowledge soundness and zero-knowledge.

We present the proof of Theorem `theo:acc-security` in Appendix A.

Complexity. We analyze the theoretical performance of the accumulation scheme. Denote by s the length of instance vector $\mathbf{x}$ and t the maximum length of messages $[\mathbf{m}_i]_{i \in [\mu]}$. We assume the commitment scheme is conducted over cyclic groups for simplicity.

- The accumulation prover asks μ queries to ρ_{NARK} , $\log m$ queries to ρ_{ACC} , and 1 query to H . Computes $E = \text{Commit}(\text{ck}, \mathbf{e})$ with n $\mathbb{G}$ operations. Performs $O(m \cdot (\mu + s + t))$ $\mathbb{F}$ operations and $m \cdot \mu$ $\mathbb{G}$ operations to accumulate predicate pairs and accumulators. Performs $O(m \cdot n \cdot d \cdot t \cdot \mu)$ $\mathbb{F}$ operations for the proving of the sum-check protocol.
- The accumulation verifier asks μ queries to ρ_{NARK} , $\log m$ queries to ρ_{ACC} , and 1 query to H . Performs $O(m \cdot \mu)$ $\mathbb{F}$ operations and $m \cdot (\mu + 1)$ $\mathbb{G}$ operations to accumulate predicate instances and accumulator instances. Performs $O(\log m \cdot n \cdot d)$ $\mathbb{F}$ for the verification of the sum-check protocol.
- The decider computes the commitments with $O(\mu \cdot t + n)$ $\mathbb{G}$ operations. Performs computations of the degree- s homogeneous algebraic map with inputs $(s + \mu \cdot t + \mu)$ -length inputs for all $s \in [d]$.

Optimizations. Note that for simplicity, we do not present some optimizations explicitly in the above accumulation schemes. Here we mention a technique mentioned in [BC23] to compress the n degree- d checks in the verification algorithm V_{sps} into one. As a result, the $n + m \cdot n$ sum-check statements can be reduced to only $1 + n$. At the final step, the verifier additionally samples a challenge $\delta \leftarrow \$ \mathbb{F}$ and the prover responds with a vector $\boldsymbol{\delta} = (\delta, \delta^2, \delta^4, \dots, \delta^{2^{\log \nu - 1}})$ (assume n is a power of 2). For $j \in [n]$, let $S \subset \{0, \dots, \log n - 1\}$ be the set such that $j = \sum_{k \in S} 2^k + 1$. The verifier derives $\delta^j = \text{pow}_j(\boldsymbol{\delta})$, where pow_j is a $\log n$ -variate polynomial. Accordingly, the prover and verifier agree on a linearly combined map with degree $D = d + \log n$ as follows,

$$\sum_{i \in [n]} \text{pow}_j(\boldsymbol{\delta}) \cdot V_j(\text{pi}, [\mathbf{m}_i]_{i=1}^\mu, [r_i]_{i=1}^\mu) = 0,$$

where $V_j(\cdot) := V_{\text{sps}}(\cdot)[j]$ is the j -th equation checked by V_{sps} .

References

- AC20. Thomas Attema and Ronald Cramer. Compressed Σ -Protocol Theory and Practical Application to Plug & Play Secure Algorithmics. In *Proc. of the Annual International Cryptology Conference (CRYPTO)*, pages 513–543. Springer, 2020.
- ACK21. Thomas Attema, Ronald Cramer, and Lisa Kohl. A compressed σ -protocol theory for lattices. In *Annual International Cryptology Conference*, pages 549–579. Springer, 2021.
- Ajt96. Miklós Ajtai. Generating hard instances of lattice problems. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 99–108, 1996.
- BBB⁺18. Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE symposium on security and privacy (SP)*, pages 315–334. IEEE, 2018.
- BC23. Benedikt Bünz and Binyi Chen. Protostar: Generic efficient accumulation/folding for special-sound protocols. In *International Conference on the Theory and Application of Cryptology and Information Security*, pages 77–110. Springer, 2023.
- BC24. Dan Boneh and Binyi Chen. Latticefold: A lattice-based folding scheme and its applications to succinct proof systems. *Cryptology ePrint Archive*, 2024.
- BC25. Dan Boneh and Binyi Chen. Latticefold+: faster, simpler, shorter lattice-based folding for succinct proof systems. In *Annual International Cryptology Conference*, pages 327–361. Springer, 2025.
- BCCT13. Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. Recursive composition and bootstrapping for snarks and proof-carrying data. In *Proceedings of the forty-fifth annual ACM symposium on Theory of computing*, pages 111–120, 2013.
- BCFW25. Benedikt Bünz, Alessandro Chiesa, Giacomo Fenzi, and William Wang. Linear-time accumulation schemes. *Cryptology ePrint Archive*, 2025.
- BCG21. Elette Boyle, Ran Cohen, and Aarushi Goel. Breaking the $o(\sqrt{n})$ -bit barrier: Byzantine agreement with polylog bits per party. In *Proceedings of the 2021 ACM Symposium on Principles of Distributed Computing*, pages 319–330, 2021.
- BCL⁺21. Benedikt Bünz, Alessandro Chiesa, William Lin, Pratyush Mishra, and Nicholas Spooner. Proof-carrying data without succinct arguments. In *Annual International Cryptology Conference*, pages 681–710. Springer, 2021.
- BCMS20. Benedikt Bünz, Alessandro Chiesa, Pratyush Mishra, and Nicholas Spooner. Proof-carrying data from accumulation schemes. *Cryptology ePrint Archive*, 2020.
- BDFG21. Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. Halo infinite: Proof-carrying data from additive polynomial commitments. In *Advances in Cryptology–CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part I 41*, pages 649–680. Springer, 2021.
- BGH19. Sean Bowe, Jack Grigg, and Daira Hopwood. Recursive proof composition without a trusted setup. *Cryptology ePrint Archive*, 2019.

- BMNW24. Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. Accumulation without homomorphism. *Cryptology ePrint Archive*, 2024.
- BMNW25. Benedikt Bünz, Pratyush Mishra, Wilson Nguyen, and William Wang. Arc: Accumulation for reed–solomon codes. In *Annual International Cryptology Conference*, pages 128–160. Springer, 2025.
- BMRS20a. Joseph Bonneau, Izaak Meckler, Vanishree Rao, and Evan Shapiro. Coda: Decentralized cryptocurrency at scale. *Cryptology ePrint Archive*, 2020.
- BMRS20b. Joseph Bonneau, Izaak Meckler, Vanishree Rao, and Evan Shapiro. Mina: Decentralized cryptocurrency at scale. *New York Univ. O (1) Labs, New York, NY, USA, Whitepaper*, pages 1–47, 2020.
- BSBHR19. Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable zero knowledge with no trusted setup. In *Annual international cryptology conference*, pages 701–732. Springer, 2019.
- BSCTV17. Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. Scalable zero knowledge via cycles of elliptic curves. *Algorithmica*, 79(4):1102–1160, 2017.
- CCDW20. Weikeng Chen, Alessandro Chiesa, Emma Dauterman, and Nicholas P Ward. Reducing participation costs via incremental verification for ledger systems. *Cryptology ePrint Archive*, 2020.
- CCYY25. Mingshu Cong, Sherman SM Chow, Siu Ming Yiu, and Tsz Hon Yuen. Scalable zkSNARKs for matrix computations: A generic framework for verifiable deep learning. *Cryptology ePrint Archive*, 2025.
- CFS17. Alessandro Chiesa, Michael A Forbes, and Nicholas Spooner. A zero knowledge sumcheck and its applications. *arXiv preprint arXiv:1704.02086*, 2017.
- Che25. Binyi Chen. Symphony: Scalable snarks in the random oracle model from lattice-based high-arity folding. *Cryptology ePrint Archive*, 2025.
- COS20. Alessandro Chiesa, Dev Ojha, and Nicholas Spooner. Fractal: Post-quantum and transparent recursive proofs from holography. In *Advances in Cryptology–EUROCRYPT 2020: 39th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zagreb, Croatia, May 10–14, 2020, Proceedings, Part I 39*, pages 769–793. Springer, 2020.
- CT10. Alessandro Chiesa and Eran Tromer. Proof-carrying data and hearsay arguments from signature cards. In *ICS*, volume 10, pages 310–331, 2010.
- CTV13. Stephen Chong, Eran Tromer, and Jeffrey A Vaughan. Enforcing language semantics using proof-carrying data. *Cryptology ePrint Archive*, 2013.
- CTV15. Alessandro Chiesa, Eran Tromer, and Madars Virza. Cluster computing in zero knowledge. In *Advances in Cryptology–EUROCRYPT 2015: 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26–30, 2015, Proceedings, Part II 34*, pages 371–403. Springer, 2015.
- Dia21. Benjamin E Diamond. Many-out-of-many proofs and applications to anonymous zether. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1800–1817. IEEE, 2021.
- EG23. Liam Eagen and Ariel Gabizon. Protogalaxy: Efficient protostar-style folding of multiple instances. *Cryptology ePrint Archive*, 2023.
- FS86. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.

- Gro16. Jens Groth. On the size of pairing-based non-interactive arguments. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 305–326. Springer, 2016.
- GWC19. Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, 2019.
- Hab22. Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. *Cryptology ePrint Archive*, 2022.
- KKC⁺25. Jiwon Kim, Abhiram Kothapalli, Orestis Chardouvelis, Riad S Wahby, and Paul Grubbs. Alpaca: Anonymous blocklisting with constant-sized updatable proofs. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3364–3382. IEEE, 2025.
- KS22. Abhiram Kothapalli and Srinath Setty. Supernova: Proving universal machine executions without universal circuits. *Cryptology ePrint Archive*, 2022.
- KS24. Abhiram Kothapalli and Srinath Setty. Hypernova: Recursive arguments for customizable constraint systems. In *Annual International Cryptology Conference*, pages 345–379. Springer, 2024.
- KST22. Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*, pages 359–388. Springer, 2022.
- LFKN92. Carsten Lund, Lance Fortnow, Howard Karloff, and Noam Nisan. Algebraic methods for interactive proof systems. *Journal of the ACM (JACM)*, 39(4):859–868, 1992.
- NDC⁺24. Wilson Nguyen, Trisha Datta, Binyi Chen, Nirvan Tyagi, and Dan Boneh. Mangrove: A scalable framework for folding-based snarks. In *Annual International Cryptology Conference*, pages 308–344. Springer, 2024.
- nex24. Nexus zkvm. <https://docs.nexus.xyz/zkvm/nexus-zkvm>, 2024. The Nexus zkVM is licensed under the source-available Business Source License (BUSL), converting to an open-source dual Apache 2.0 and MIT licensing on February 10th, 2029.
- Set20. Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In *Annual International Cryptology Conference*, pages 704–737. Springer, 2020.
- STW23. Srinath Setty, Justin Thaler, and Riad Wahby. Customizable constraint systems for succinct arguments. *Cryptology ePrint Archive*, 2023.
- Val08. Paul Valiant. Incrementally verifiable computation or proofs of knowledge imply time/space efficiency. In *Theory of cryptography conference*, pages 1–18. Springer, 2008.
- XZZ⁺19. Tiacheng Xie, Jiaheng Zhang, Yupeng Zhang, Charalampos Papamanthou, and Dawn Song. Libra: Succinct zero-knowledge proofs with optimal prover computation. In *Advances in Cryptology–CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part III 39*, pages 733–764. Springer, 2019.
- ZGGX23. Tianyu Zheng, Shang Gao, Yu Guo, and Bin Xiao. Kilonova: Non-uniform pcd with zero-knowledge property from generic folding schemes. *Cryptology ePrint Archive*, 2023.
- ZGGX25. Tianyu Zheng, Shang Gao, Yu Guo, and Bin Xiao. quasar: sublinear accumulation schemes for multiple instances. *Cryptology ePrint Archive*, 2025.

- ZGSX23. Tianyu Zheng, Shang Gao, Yubo Song, and Bin Xiao. Leaking arbitrarily many secrets: Any-out-of-many proofs and applications to ringct protocols. In *2023 IEEE Symposium on Security and Privacy (SP)*, pages 2533–2550. IEEE, 2023.
- ZZD23. Zibo Zhou, Zongyang Zhang, and Jin Dong. Proof-carrying data from multi-folding schemes. *Cryptology ePrint Archive*, 2023.

A Security Proof of Theorem 3

In this part, we provide the proofs for the security properties of the proposed accumulation schemes. Since the proof of completeness and knowledge soundness follows the general idea in previous works [BC23], we omit some details for simplicity.

Completeness. Assume the inputs including m satisfying predicate instance-proof pairs $[\mathbf{qx}_j, \mathbf{qw}_j]_{j=1}^m$ and m satisfying accumulators $[\mathbf{acc}_j]_{j=1}^m$. Let $\mathbf{acc}, \mathbf{pf}$ denote the output of the accumulation prover ACC.P. We argue that both the accumulation verifier and the decider will accept, i.e.,

$$\mathbf{acc.V}^{\text{RO}}(\mathbf{avk}, [\mathbf{qx}_i]_{i=1}^n, [\mathbf{acc}_j.\mathbf{x}]_{j=1}^m, \mathbf{acc.x}, \mathbf{pf}) = 1 \quad \mathbf{acc.D}(\mathbf{adk}, \mathbf{acc}) = 1$$

For ACC.V, as he honestly computes the linear combination, the obtained $\mathbf{acc.x}([\mathbf{r}_i]_{i=1}^{\mu-1}, [\mathbf{C}_i]_{i=1}^{\mu}, E, \mathbf{v}_g)$ is consistent as the output of the prover.

For the decider ACC.D($\mathbf{adk}, \mathbf{acc}$) with input accumulator

$$\mathbf{acc} = (\mathbf{acc.x} = (h, [\mathbf{C}_i]_{i=1}^{\mu}, [\mathbf{r}_i]_{i=1}^{\mu-1}, E, \mathbf{\beta}, \mathbf{v}_g), \mathbf{acc.w} = (\mathbf{x}, [\mathbf{m}_i]_{i=1}^{\mu}, \mathbf{G}(\mathbf{X})),$$

we argue that:

1. $\mathbf{acc.C}_i = \text{Commit}(\mathbf{ck}, \mathbf{m}_i)$ for all $i \in [\mu]$ due to the homomorphism of the commitment scheme.
2. the computed $\mathbf{e} \leftarrow \sum_{k=0}^d f_k^{\text{VSPS}}(\mathbf{x}, [\mathbf{m}_i]_{i=1}^{\mu}, [\mathbf{r}_i]_{i=1}^{\mu-1})$ satisfies that

$$\mathbf{e} := \mathbf{F}(\mathbf{\beta}) = \sum_{k=0}^d f_k^{\text{VSPS}}(\mathbf{x}(\mathbf{\beta}), [\mathbf{m}_i(\mathbf{\beta})]_{i=1}^{\mu}, [\mathbf{r}_i(\mathbf{\beta})]_{i=1}^{\mu-1}).$$

Again $E = \text{Commit}(\mathbf{ck}, \mathbf{e})$ holds by the homomorphism of the commitment scheme.

3. $\mathbf{G}(\mathbf{\beta}) = \mathbf{v} = \mathbf{g}$ because it is the linear combination of the $m + 1$ equations

$$\mathbf{G}'(\mathbf{\beta}) = \mathbf{v}'_g, \quad \mathbf{acc}_j.\mathbf{w}.\mathbf{G}(\mathbf{\beta}) = \mathbf{v}'_{g,j} \forall j \in [m].$$

Knowledge Soundness. We prove that the accumulation scheme has knowledge-soundness by showing that there exists an underlying $(2 \log m + 5)$ -move interactive proof with $(2, 2, \dots, 2)$ -special-soundness. Consider the public-coin interactive proof $\Pi_I = (\mathbf{P}_I, \mathbf{V}_I)$ where $\mathbf{P}_I$ first sends image h and $\mathbf{V}_I$ responds with challenges α, γ . Next, $\Pi_I, \mathbf{V}_I$ engage in a $2 \log m + 3$ -move sum-check protocol interactively, where Π_I sends $\mathbf{F}_{\log m+1}(X), \mathbf{v}, [\mathbf{v}'_{g,j}]_{j=1}^m, \mathbf{v}'_g$ at the final move,

Extracting witness. We first show that given $2^{\log m+2} = 4m$ accepting transcripts with distinct challenges, there exists an extractor outputting $[\mathbf{qw}_j]_{j=1}^m, [\mathbf{acc}_j.\mathbf{w}]_{j=1}^m$. Simply, given $4m$ accepting transcripts with distinct γ, β , the extractor can compute the following

$$\begin{aligned} \mathbf{qw}_j.\mathbf{m}_i, \forall i \in [\mu], j \in [0, m], \quad \mathbf{acc}_j.\mathbf{w}.\mathbf{x}, \forall j \in [0, m], \\ \mathbf{acc}_j.\mathbf{w}.\mathbf{m}_i, \forall i \in [\mu], j \in [0, m], \quad \mathbf{acc}_j.\mathbf{w}.\mathbf{G}(\mathbf{X}), \forall j \in [0, m]. \end{aligned}$$

Otherwise, the extractor will break the binding property of the commitment scheme by finding a collision for one of the linear combinations below, where $i \in [\mu]$:

$$\mathbf{acc}.\mathbf{x}.C_i = \mathbf{eq}_0(\beta) \cdot \mathbf{qx}_0.C_i + \sum_{j=1}^m [\mathbf{eq}_{j-1}(\beta) \cdot \mathbf{qx}_j.C_i + \mathbf{eq}_{j+m-1}(\beta) \cdot \mathbf{acc}_j.\mathbf{x}.C_i],$$

which is implied by the precondition: $\text{ACC.V}(\text{avk}, [\mathbf{qx}_j, \mathbf{acc}_j.\mathbf{x}]_{j=1}^m, \mathbf{acc}.\mathbf{x}, \text{pf}) = 1$.

Ensuring Predicates. Additionally, the extractor needs to ensure that all the extracted witnesses satisfy the constraint V_{sps} . Since both ACC.V and ACC.D pass, the extracted witness satisfies except for a negligible probability:

$$\begin{aligned} \mathbf{eq}(\beta, \alpha) \cdot \mathbf{F}(\beta) + \gamma \cdot \mathbf{v}'_g &= \mathbf{v} \\ \mathbf{acc}_j.\mathbf{w}.\mathbf{G}(\beta) &= \mathbf{v}'_g \forall j \in [m] \end{aligned}$$

Then ensured by the soundness of the sum-check protocol, we have the following

$$\begin{aligned} \sum_{\mathbf{b} \in \{0,1\}^{\log m+1}} [\mathbf{eq}(\mathbf{b}, \alpha) \cdot \mathbf{F}(\mathbf{b}) + \gamma \cdot \mathbf{G}'(\mathbf{b})] &= \sum_{\mathbf{b} \in \{0,1\}^{\log m+1}} \mathbf{eq}(\mathbf{b}, \alpha) \cdot \mathbf{e}(\mathbf{b}) + \gamma \cdot \mathbf{e}'_g \in \mathbb{F}^n. \\ \sum_{\mathbf{b} \in \{0,1\}^{\log m+1}} \mathbf{eq}(\mathbf{acc}_j.\mathbf{x}.\beta, \mathbf{b}) \cdot \mathbf{acc}_j.\mathbf{w}.\mathbf{G}(\mathbf{b}) &= \mathbf{acc}_j.\mathbf{x}.\mathbf{v}_g \forall j \in [m] \end{aligned}$$

Then guaranteed by the Schwartz-Zippel lemma, the extracted witness satisfies the following constraints:

$$\begin{aligned} \sum_{k=0}^d f_k^{\text{V}_{\text{sps}}}(\mathbf{qx}_j.\mathbf{x}, [\mathbf{qw}_j.\mathbf{m}_i]_{i=1}^{\mu}, [\mathbf{qx}_j.r_i]_{i=1}^{\mu-1}) &= \mathbf{0}^n, \forall j \in [m], \\ \sum_{k=0}^d f_k^{\text{V}_{\text{sps}}}(\mathbf{acc}_j.\mathbf{w}.\mathbf{x}, [\mathbf{acc}_j.\mathbf{w}.\mathbf{m}_i]_{i=1}^{\mu}, [\mathbf{acc}_j.\mathbf{x}.r_i]_{i=1}^{\mu-1}) &= \mathbf{acc}_j.\mathbf{w}.\mathbf{e}, \\ \mathbf{acc}_j.\mathbf{x}.E &= \text{Commit}(\text{ck}, \mathbf{acc}_j.\mathbf{w}.\mathbf{e}) \\ \mathbf{acc}_j.\mathbf{x}.\mathbf{v}_g &= \mathbf{acc}_j.\mathbf{w}.\mathbf{G}(\beta). \end{aligned}$$

Otherwise, the extractor finds a collision for the following linear combination:

$$\mathbf{acc}_j.\mathbf{x}.E = \sum_{j=1}^m \mathbf{eq}_{j+m-1}(\beta) \cdot \mathbf{acc}_j.\mathbf{x}.E.$$

Zero-Knowledge. Consider an adversary $\mathcal{A}$ outputting $(i_\Phi[(q\mathbf{x}_j, q\mathbf{w}_j)]_{j=1}^m, [\mathbf{acc}_j]_{j=1}^m)$, such that $\Phi(\mathbf{pp}_\Phi, i_\Phi, q\mathbf{x}_j, q\mathbf{w}_j) = 1$ and $\text{ACC.D}(\mathbf{adk}, \mathbf{acc}_j) = 1$ for all $j \in [m]$. We show that it is efficient to build a simulator Sim with inputs $(\tau = \perp, \mathbf{pp}_\Phi, i_\Phi, [q\mathbf{x}_i]_{i=1}^n, [\mathbf{acc}_j]_{j=1}^m)$, outputting an accumulator $\mathbf{acc}^*$ and a proof $\mathbf{pf}^*$ that are indistinguishable from the outputs by an honest prover.

Concretely, given inputs as $\mathbf{pp}_\Phi, i_\Phi$, the simulator first sample the elements in $\mathbf{acc}^*$ as $\mathbf{x}^* \leftarrow \mathbb{F}^k$, $\mathbf{m}_i^* \leftarrow \mathbb{F}^t \forall i \in [\mu]$, $r_i^* \leftarrow \mathbb{F} \forall i \in [\mu - 1]$ and $\beta \leftarrow \mathbb{F}^{\log m + 1}$. Next, the simulator simulates the accumulation proof $\mathbf{pf}^*$ consists of a random predicate instance $q\mathbf{x}_0$, a sum-check transcript $\mathbf{tr}^*$, and a vector $(\mathbf{v}'_g)^*$. For $q\mathbf{x}_0$, the simulator computes

$$\begin{aligned} q\mathbf{x}_0.r_i &:= eq_0^{-1}(\beta) \cdot \left[r_i^* - \sum_{j=1}^m (eq_{j-1}(\beta) \cdot q\mathbf{x}_j.r_i + eq_{j+m-1}(\beta) \cdot \mathbf{acc}_j.x.r_i) \right], \forall i \in [\mu - 1] \\ q\mathbf{x}_0.C_i &:= eq_0^{-1}(\beta) \cdot \left[C_i^* - \sum_{j=1}^m (eq_{j-1}(\beta) \cdot q\mathbf{x}_j.C_i + eq_{j+m-1}(\beta) \cdot \mathbf{acc}_j.x.C_i) \right], \forall i \in [\mu] \end{aligned}$$

where $C_i^* := \text{Commit}(\text{ck}, \mathbf{m}_i^*)$. The predicate instance vector $q\mathbf{x}_0.\mathbf{x}$ can be randomly sampled from $\mathbb{F}^k$.

For the sum-check proof, the simulator randomly samples two polynomial vectors $\mathbf{F}^*(\mathbf{X}), (\mathbf{G}'(\mathbf{X}))^* \in (\mathbb{F}^{(\leq d)}[\mathbf{X}])^n$ satisfying the following evaluation claims:

$$\mathbf{F}^*(\beta) = \mathbf{e} \wedge (\mathbf{G}'(\beta))^* = \mathbf{v}'_g$$

Then the simulator Sim runs the simulator Sim_{sc} of the zero-knowledge protocol with inputs of the above evaluation claims, and obtains a simulated proof $\mathbf{tr}$ for the following statement:

$$\begin{aligned} \sum_{\mathbf{b} \in \{0,1\}^{\log m + 1}} [eq(\mathbf{b}, \alpha) \cdot \mathbf{F}(\mathbf{b}) + \gamma \cdot \mathbf{G}'(\mathbf{b})] &= \sum_{\mathbf{b} \in \{0,1\}^{\log m + 1}} eq(\mathbf{b}, \alpha) \cdot \mathbf{e}(\mathbf{b}) + \gamma \cdot \mathbf{e}'_g \in \mathbb{F}^n. \\ \sum_{\mathbf{b} \in \{0,1\}^{\log m + 1}} eq(\mathbf{acc}_j.x, \beta, \mathbf{b}) \cdot \mathbf{acc}_j.w.\mathbf{G}(\mathbf{b}) &= \mathbf{acc}_j.x.\mathbf{v}_g \forall j \in [m] \end{aligned}$$

Note that to ensure the simulated sum-check proof is accepted by the verifier, the simulator needs to program the random oracle ρ_{NARK} by writing each query-response pair $[(\mathbf{F}_j(\mathbf{X}), \beta_{j-1}), \beta_j], \forall j \in [\log m + 1]$.

After that, the simulator constructs the accumulator $\mathbf{acc}^*$ by computing the remaining elements accordingly,

$$\begin{aligned} h &:= H(\mathbf{x}), C_i := \text{Commit}(\text{ck}, \mathbf{m}_i) \forall i \in [\mu] \\ E &:= \text{Commit}(\text{ck}, \mathbf{e}), \mathbf{v}_g = eq_0(\beta) \cdot \mathbf{v}'_g + \sum_{j=1}^m eq_{j+m-1}(\beta) \cdot \mathbf{v}'_{g,j} \\ \mathbf{acc}.x &:= (h, [C_i]_{i=1}^\mu, [r_i]_{i=1}^{\mu-1}, E, \beta, \mathbf{v}_g), \mathbf{acc}.w := (\mathbf{x}, [\mathbf{m}_i]_{i=1}^\mu, \mathbf{G}(\mathbf{X})) \end{aligned}$$

Clearly, the accumulator acc^* is accepted by the decider ACC.D . By writing the following query-response pairs into the random oracle ρ_{NARK} , the pair $(\text{acc}^*, \text{pf}^*)$ is accepted by the accumulation verifier ACC.V .

- Writes the pair $[\text{qx}_0.\mathbf{x}, \text{qx}_0.r_1] \text{qx}_j.r_0 = \rho_{\text{NARK}}(\text{qx}_j.\mathbf{x})$.
- Writes the pair $[(\text{qx}_0.r_{i-1}, \text{qx}_0.C_i), \text{qx}_0.r_i]$ for all $i \in [\mu - 1]$.
- Writes the pair $[(\text{acc}.\mathbf{x}^*), h^*]$.

Finally, we show that acc^* is indistinguishable from the acc generated in the honest case by $\text{ACC.P}^{\text{RO}}(\text{apk}, [(\text{qx}_j, \text{qw}_j)]_{j=1}^m, [\text{acc}_j]_{j=1}^m)$ because the random predicate pair $(\text{qx}_0, \text{qw}_0)$ are chosen uniformly at random from a distribution, as the random linear combination between $(\text{qx}_0, \text{qw}_0)$ and other inputs $[(\text{qx}_j, \text{qw}_j)]_{j=1}^m, [\text{acc}_j]_{j=1}^m$, the resulted $(\text{acc}.\mathbf{x}, \text{acc}.\mathbf{w})$ also satisfies the distribution.

B Related Works

Prior to the development of accumulation and folding schemes, the only known method for constructing PCD was through recursive composition of succinct non-interactive arguments (SNARGs) [BCCT13, CTV15, COS20]. In this approach, a proof for the correctness of t computation steps consists of a proof for the claim: “the t -th step was executed correctly, and there exists a proof that the computation was executed correctly for $t - 1$ steps.” The latter part is encoded using the SNARG verifier itself. This construction yields secure PCD (with IVC as a special case) provided the SNARG satisfies adaptive knowledge soundness (i.e., is a SNARK). The efficiency and security of the resulting PCD scheme correspond to those of a single invocation of the SNARK.

In 2019, Halo [BGH19] and subsequent works [BDFG21, BCMS20, BCL⁺21] introduced a practical approach to deferring expensive cryptographic operations in proof verification by batching them at the end of IVC/PCD protocols. Instead of verifying SNARK or (non-succinct) NARK proofs from previous steps within the recursive circuit, the prover employs an *accumulation scheme* to defer these checks and continue incremental computation. Specifically, an accumulation scheme enables the prover to convince the verifier that multiple instances can be linearly combined into a single *running instance*, whose verification is significantly more efficient than directly checking all previous proofs. By recursively accumulating at each step, the protocol produces a final instance representing the entire computation, with size comparable to a single step, which is then verified by the IVC/PCD verifier.

More recently, Nova [KST22] introduced a highly efficient IVC construction based on *folding schemes*. Unlike Halo, where the prover accumulates SNARK/NARK proofs compiled with polynomial commitment schemes (PCS) such as KZG or Bulletproofs [BBB⁺18], Nova’s approach applies folding schemes—such as random linear combinations—directly to NP *instances* derived from Interactive Oracle Proofs (IOP), without requiring PCS compilation. This distinction leads to substantial performance improvements, as verifying uncompiled NP instances involves fewer cryptographic operations than verifying proofs. Subsequent works,

including SuperNova [KS22], Protostar [BC23], Protogalaxy [EG23], HyperNova [KS24] and KiloNova [ZGGX23] have further developed these ideas. As the distinction between “folding” and “accumulation” has become less pronounced in recent literature, we use these terms interchangeably in this paper.

Lattice-based accumulation schemes can leverage Ajtai’s partially homomorphic commitments [Ajt96]. The main challenge in this setting is controlling the norm growth of the committed (and extracted) witness, which is essential for enabling unbounded accumulation. Recent works [BC24, BC25, Che25] have made significant progress in constructing lattice-based accumulation schemes that address this challenge. However, to the best of our knowledge, no existing construction achieves a linear-time prover, due either to superlinear commitment costs or superlinear complexity in the accumulation prover. Achieving a lattice-based accumulation scheme that controls norm growth while maintaining a linear-time prover remains an open problem.

Additionally, some recent constructions in the random oracle model (ROM) utilize Merkle commitment schemes [BMNW24, BCFW25, ZGGX25], but their provers run in quasilinear time due to the use of Reed–Solomon codes and polynomial quotients.

C Definitions

C.1 Commitment Schemes

A commitment scheme is a pair of PPT algorithms ($\text{Gen}, \text{Commit}$) [AC20].

- $\text{ck} \leftarrow \text{Gen}(1^\lambda)$: on input security parameter λ , outputs a commitment key ck , where ck specifies a message space $\mathcal{M}_c$, a key space $\mathcal{K}_c$, a commitment space $\mathcal{C}_c$, a randomness space $\mathcal{R}_c$ and an opener space $\mathcal{O}_c$.
- $c_m \leftarrow \text{Commit}(\text{ck}; m)$: on input commitment key $\text{ck} \in \mathcal{K}_c$, a message $m \in \mathcal{M}_c$, a randomness $r \in \mathcal{R}_c$, outputs a commitment c_m . Parameter r can be omitted when the sampling is implicit.

Typically, a commitment scheme should satisfy binding and hiding properties defined in Appendix C.1.

Definition 11. (Binding Property) *A non-interactive commitment scheme (Gen, Com) is computationally binding if for all PPT adversaries $\mathcal{A}$, there is a negligible function $\mu(\lambda)$ such that*

$$\Pr \left[\begin{array}{l} \text{Commit}(\text{ck}; m_0; r_0) \\ = \text{Commit}(\text{ck}; m_1; r_1) \\ \wedge m_0 \neq m_1 \end{array} \middle| \begin{array}{l} \text{ck} \leftarrow \text{Gen}(1^\lambda); \\ (m_0, m_1, r_0, r_1) \leftarrow \mathcal{A}(\text{ck}) \end{array} \right] = \mu(\lambda).$$

where $\mathcal{A}$ outputs $m_0, m_1 \in \mathcal{M}_c$ and $r_0, r_1 \in \mathcal{R}_c$. The scheme is **perfectly binding** when $\mu(\lambda) = 0$.

Definition 12. (Hiding Property *A non-interactive commitment scheme* $(\text{Gen}, \text{Commit})$ is computationally hiding if for all PPT adversaries $\mathcal{A}$, there is an negligible function $\mu(\lambda)$ such that

$$\left| \Pr \left[\mathcal{A}(c) = b \mid \begin{array}{l} \text{ck} \leftarrow \text{Gen}(1^\lambda); \\ (m_0, m_1) \leftarrow \mathcal{A}(\text{ck}), \\ b \leftarrow \{0, 1\}, r \leftarrow \mathcal{R}_c, \\ c \leftarrow \text{Commit}(\text{ck}; m_b; r) \end{array} \right] - \frac{1}{2} \right| = \mu(\lambda),$$

where $\mathcal{A}$ outputs $m_0, m_1 \in \mathcal{M}$. The scheme is **perfectly hiding** when $\mu(\lambda) = 0$.

C.2 Definitions for Special Sound Protocols

For an $\mathcal{NP}$ relation $\mathcal{R} := \{(x; w)\}$, an interactive proof for $\mathcal{R}$ allows a prover P to convince a verifier V that a statement x admits a witness w . Typically, in a $(2\mu - 1)$ -move protocol, the prover and the verifier interactively send μ messages and $\mu - 1$ challenges. If V generates and sends all its messages uniformly at random and independently of the prover's messages, then the protocol is called a *public coin* protocol. Public coin protocols can be transformed into non-interactive forms according to the Fiat-Shamir heuristic [FS86]. For security definitions, we require the interactive proof to satisfy completeness and knowledge soundness. Furthermore, to simplify the security proof, researchers may refer to the *special soundness* definition that implies knowledge soundness (see Lemma 3). Briefly speaking, an interactive proof satisfies special soundness if there exists an efficient extractor outputting a valid witness with inputs of the instance x and a tree of transcripts, which is a collection of accepting transcripts generated by a malicious prover, where the challenges differ in a specific pattern across the transcripts. For defining the special soundness property, we first present a denotation of the tree of transcript.

Definition 13 (Tree of transcripts). Let $k \in \mathbb{N}$ and $(a_1, \dots, a_k) \in \mathbb{N}^k$. A $(a_1, \dots, a_k)$ -tree of transcripts constitutes a set of $\prod_{i=1}^k a_i$ transcripts of a tree-like structure. The edges within this tree represent the challenges of the verifier, while the vertices are the messages from the prover, which can be empty. Each node at depth i has a_i child nodes, corresponding to a_i distinct challenges. Every transcript is uniquely represented by one path from the root node to a leaf node.

Definition 14 $((a_1, \dots, a_k)$ -special soundness). Π provides $(a_1, \dots, a_k)$ -special soundness, if there is an effective PPT extraction algorithm $\mathcal{E}$ that is capable of extracting the witness w given x and any $(a_1, \dots, a_k)$ -tree of accepting transcripts T [ACK21] (defined above). Specifically, for all PPT adversaries $\mathcal{A}$

$$\Pr [\text{pp} \leftarrow G(1^\lambda), (x, T) \leftarrow \mathcal{A}(\text{pp}), w \leftarrow \mathcal{E}(\text{pp}, x, T) : (x; w) \in \mathcal{R}] \approx 1.$$

According to the conclusion given in [AC20], we can imply knowledge soundness from special soundness:

Lemma 3 $((a_1, \dots, a_k)$ -special soundness implies knowledge soundness [ACK21]).

Let $k, a_1, \dots, a_k \in \mathbb{N}$, (G, P, V) be a $(a_1, \dots, a_k)$ -special sound $(2k + 1)$ -move interactive protocol for relation $\mathcal{R}$, where V samples each challenge uniformly at random from $\mathbb{F}$. Then (G, P, V) is knowledge sound with knowledge error

$$\kappa \leq \frac{\sum_{i=1}^k a_i - 1}{|\mathbb{F}|}.$$

C.3 Security Definition for NARK

Completeness. NARK is complete if for every (unbounded) adversary $\mathcal{A}$,

$$\Pr \left[\begin{array}{c} (i, x, w) \notin R(pp) \\ \vee \\ \text{NARK.V}^{\text{RO}}(ivk, x, \pi) = 1 \end{array} \middle| \begin{array}{c} \text{RO} \leftarrow \mathcal{U}(\lambda) \\ pp \leftarrow \text{NARK.G}^{\text{RO}}(1^\lambda) \\ (i, x, w) \leftarrow \mathcal{A}^{\text{RO}}(pp) \\ (ipk, ivk) \leftarrow \text{NARK.I}^{\text{RO}}(pp, i) \\ \pi \leftarrow \text{NARK.P}^{\text{RO}}(ipk, x, w) \end{array} \right] = 1.$$

Knowledge soundness. NARK is knowledge sound with respect to auxiliary input distribution Ψ if for every expected polynomial-time adversary $\tilde{P}$, there exists an expected polynomial-time extractor $\mathcal{E}$ such that for every set Z ,

$$\begin{aligned} & \Pr \left[\begin{array}{c} (pp, ai, \mathbf{i}, \mathbf{x}, ao) \in Z \\ \wedge \\ \forall j \in [\ell], \text{NARK.V}^{\text{RO}}(ivk_j, x_j, w_j) = 1 \end{array} \middle| \begin{array}{c} \text{RO} \leftarrow \mathcal{U}(\lambda) \\ pp \leftarrow \text{NARK.G}(1^\lambda) \\ ai \leftarrow \Psi(pp) \\ (\mathbf{i}, \mathbf{x}, \pi, ao) \leftarrow \tilde{P}^{\text{RO}}(pp, ai) \\ \forall j \in [\ell], (ipk_j, ivk_j) \leftarrow \text{NARK.I}^{\text{RO}}(pp, i_j) \end{array} \right] \\ & \leq \Pr \left[\begin{array}{c} (pp, ai, \mathbf{i}, \mathbf{x}, ao) \in Z \\ \wedge \\ \forall j \in [\ell], (i_j, x_j, w_j) \in R(pp) \end{array} \middle| \begin{array}{c} \text{RO} \leftarrow \mathcal{U}(\lambda) \\ pp \leftarrow \text{NARK.G}(1^\lambda) \\ ai \leftarrow \Psi(pp) \\ (\mathbf{i}, \mathbf{x}, \mathbf{w}, ao) \leftarrow \mathcal{E}^{\tilde{P}}(pp, ai) \end{array} \right] + \text{negl}(\lambda). \end{aligned}$$

We omit the random oracle if the context is clear. For more details, we encourage the readers to refer to the definitions in [COS20].